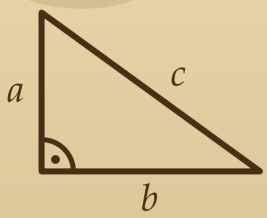
THE GNU APL BIBLE

Implementing APL in C++



$$a^2 + b^2 = c^2$$

```
      R←20 ⍝ prime number limit
      ⎕←(~R∈R∘.×R)/R←1↓⍳R
 2 3 5 7 11 13 17 19
```

DR. JÜRGEN SAUERMANN

GNU · savannah.gnu.org/projects/apl

Table of Contents

Preface 7

A Note on Source Line Numbers and Names 7

About the Exercises 9

Part I: Context and Setup 10

1. What Is GNU APL? 10

1.1. A Programming Language 10

1.2. The ISO Standard 10

1.3. The GNU Project and GNU APL 14

1.4. What Makes APL Unusual 15

1.5. Notes 16

1.6. Exercises 16

2. Building and Developing 18

2.1. Prerequisites 18

2.2. Obtaining the Source 19

2.3. The Autotools Build System 19

2.4. Key Configure Options 21

2.5. Out-of-Tree Builds 22

2.6. Definition Files 22

2.7. The Development Cycle 26

2.8. Notes 27

2.9. Exercises 27

3. The Test Infrastructure 28

3.1. Test File Format 28

3.2. Running the Test Suite 31

3.3. The Summary Log 33

3.4. Wildcards in Expected Output 33

3.5. Conditional Skipping:]NEXTFILE 35

3.6. Declaring Expected Errors:]EXPECT 36

3.7. Regenerating Expected Output 37

3.8. □RVAL and Property-Based Fuzzing 38

3.9. Profiling and Coverage Testing 43

3.10. Performance Probes 46

3.11. Notes 51

3.12. Excursus: Testing in the Large 52

3.13. Excursus: Interpreter Overhead, Then and Now 58

3.14. Exercises 59

Part II: The Core Data Model 61

4. Values, Shapes, and Ravels 61

5. Cell Types 87

6. Memory Management 92

Part III: The Execution Pipeline 97

7. Tokenization 97

8. Parsing and Executables 104

9. The State Indicator 123
10. The REPL and Command Loop 131
Part IV: Built-in Functions and Operators 138
11. Scalar Functions 138
12. Non-Scalar Functions 147
13. Operators 157
14. Quad Functions 167
Part V: Workspace and Symbol Management 180
15. Symbols and the Symbol Table 181
16. System Variables 191
17. Workspace Serialization 203
Part VI: Putting It All Together 214
18. Why `A←42` Looks Simple But Is Not 214
Part VII: Advanced and Optional Features 223
19. The libapl Embedding API 223
20. Native Functions and Auxiliary Processors 232
21. The DATIS-P Parallel APL Machine 241
22. Parallel Execution 251
23. WebSocket and GUI Interfaces 259
24. The Windows Port 268
Part VIII: Maintenance and Contribution 275
25. Infrastructure 275
26. Navigating the Source Tree 285
27. Common Maintenance Tasks 294
28. Debugging Techniques 302
29. The Contribution Workflow 311
Appendix A: Reference 315

Preface

This book is a guided tour of GNU APL's implementation, written for the audience most likely to actually need one. That audience is maintainers picking up this codebase for the first time, C++ developers curious what a real, actively-used interpreter for a genuinely unusual language looks like from the inside, and APL programmers who have called ⎕FIO or ⎕GTK from the outside and want to know what happens on the other side of the call. It assumes working C++ knowledge — templates, virtual dispatch, the STL — but assumes nothing about APL beyond what Chapter 1 introduces. Conversely, it assumes nothing about compiler or interpreter construction that the chapters on tokenization, parsing, and evaluation do not build up from first principles.

The chapters are ordered the way a new maintainer would actually need them. Build the thing first (Part I), understand the data it operates on (Part II), then the pipeline that turns typed APL into a computed result (Part III) — before the built-in functions, workspace machinery, and optional interfaces that make up the bulk of the remaining source tree. The last part turns from explaining the existing code to explaining how to change it — navigating the tree, the recurring shapes of a maintenance task, debugging technique, and the mechanics of sending a patch back upstream.

Like the interpreter it describes, this book is a work in progress. GNU APL is under active development, and a chapter finished today may cite a line number that has moved by the time it is read — see the note below. Thanks are due to the GNU Project, for giving a free implementation of an ISO-standard language a home, and to the wider APL community whose bug reports, questions, and decades of prior implementations shaped the interpreter this book describes.

A Note on Source Line Numbers and Names

A handful of places in this book cite a specific line number in a specific source file (for example, "the **`check_next_binding()`** call on line 392 of **`Prefix.cc`**", Chapter 8) rather than just naming the function or file. Every such citation — and, more broadly, every function, class, and variable name used anywhere in this book — pertains to a fixed snapshot of the GNU APL source tree, tagged **`Bible`** in the Savannah repository's **`tags`** directory alongside the tag for each numbered

release. Unlike **trunk**, which advances with every commit, a tag never changes, so **Bible** will always mean exactly the source tree this book was written against. To have that exact tree at hand while reading, check it out by tag name rather than from **trunk**:

```
svn checkout svn://svn.savannah.gnu.org/apl/tags/Bible gnu-apl-bible
```

or, from the git mirror (Chapter 29 describes how the two are kept in sync):

```
git clone https://git.savannah.gnu.org/git/apl.git
cd apl
git checkout Bible
```

GNU APL's development continues past this snapshot; by the time you read this, **trunk** may have moved a cited function to a different line, renamed it, or moved it to a different file entirely. Checking out **Bible** itself sidesteps that entirely — but if you are instead reading against a newer checkout, treat every citation, name included, as "true at the **Bible** tag, and a good starting point for **grep**," not as a permanent coordinate. Names drift far less often than line numbers, but the **Bible** tag is what makes either kind of citation checkable with certainty.

This same source tree feeds a second, complementary reference this book points to repeatedly: the Doxygen-generated API documentation covered in "Doxygen Documentation" near the end of the book. Where this book is a top-down tour — motivating why a class or file exists and following the order the interpreter itself imposes — Doxygen is the bottom-up counterpart, an alphabetized, cross-linked index of every member, every call, and every inheritance edge. Worth having open alongside the checked-out source tree itself; that section explains how to generate it.

About the Exercises

Each chapter ends with a set of exercises. Their difficulty is rated on the scale introduced by Donald Knuth in *The Art of Computer Programming*:

Table 1. Exercise difficulty ratings

Rating	Meaning
`[10]`	Immediate — about one minute
`[20]`	Simple — about a quarter of an hour
`[30]`	Moderate — several hours of work
`[40]`	Hard — a full day or more
`[50]`	Research problem — no known solution at time of writing

Most exercises in this book fall in the `[10]`–`[30]` range. An exercise rated `[15]` or `[25]` sits between the two bracketing levels.

Part I: Context and Setup

1. What Is GNU APL?

GNU APL is a free, standards-conforming implementation of APL — **ISO/IEC 13751**, *Programming Language APL, Extended*, specifically, the revision of the standard that added nested arrays to Iverson's original notation. Begun in 2008 and still under active development, it compiles to a single executable, **`apl`**, of roughly 175,000 lines of C++ (Chapter 26). It is distributed under the GNU General Public License as part of the GNU Project. None of those facts, on their own, explains much. This chapter exists to answer three questions, before the rest of the book turns to reading the codebase itself. **Why** does an array language need an interpreter built the particular way this one is? **What** does conformance to an ISO standard actually commit an implementation to? And **how** does a language whose entire user-visible surface is barely a hundred glyphs come to occupy a codebase this large?

1.1. A Programming Language

In 1962 Kenneth E. Iverson published *A Programming Language*, a notation for describing algorithms with mathematical precision. Iverson was employed by IBM at the time, and IBM subsequently implemented the notation as an interpreter running on their mainframes. That made APL one of the earliest interactive programming languages available on large-scale hardware.

1.2. The ISO Standard

APL has been standardised twice at the international level. The first standard, **ISO 8485:1989**, covered the original flat-array APL. The second and current standard, **ISO/IEC 13751:2001** (*Programming Language APL, Extended*), adds nested arrays — arrays whose elements may themselves be arrays of any shape and type. Nested arrays are the feature that gives modern APL its expressive power for handling irregular and hierarchical data.

GNU APL targets **ISO/IEC 13751**. The title of that standard is the direct inspiration for the subtitle of this book: *implementing* the interpreter means implementing the standard, and understanding the interpreter means understanding what the standard requires and why.

1.2.1. The Standard as a Virtual Machine

IBM's *APL2 Language Reference Manual* describes APL2 in the style of a conventional language reference: syntax rules, semantics of each primitive, and data types. **ISO/IEC 13751** takes a fundamentally different approach. It specifies APL not as a language definition but as a *formal model of an APL machine* (§1, Introduction). That model is a collection of objects, diagrams, and *evaluation sequences* — formal procedures, written in a structured English, that define the machine's behaviour step by step.

The standard says so explicitly:

> A user interacts with an APL system through a session, an abstraction that represents a hypothetical machine capable of carrying out the evaluation sequences in this International Standard.
>
> *§5.3.8 Sessions*
> — ISO/IEC 13751

Table 2. **ISO 13751** vs. CPU/VM analogues vs. GNU APL

ISO 13751 concept	CPU / VM analogue	GNU APL
Session	The processor itself	**src/apl** process
Evaluation sequence	Microcode / instruction semantics	C++ method bodies
Token	Opcode (with operand field)	**class Token** (**src/Token.hh**)
Token class	Opcode type (ALU, branch, …)	**TokenClass** (a sub-field of **TokenTag**)
Token-Diagram / statement analysis	Instruction decoder	**src/Parser.cc** and **src/Prefix.cc**
Workspace	Process address space	**class Workspace** (**src/Workspace.hh**)
Symbol Table	Named memory / register file	**class SymbolTable** (**src/SymbolTable.hh**)
State Indicator	Call stack	**src/StateIndicator.hh**
Context	Stack frame	**class StateIndicator**
Current-Line-Number (**I**)	Program counter (PC)	**Function_Line StateIndicator::get_line()**
Evaluate-Line / -Statement	Fetch–decode–execute cycle	**Prefix::reduce_body()**
Branch token (**→N**)	Jump instruction	**TOK_BRANCH_INT, TOK_BRANCH_LAB**
Escape token (**→**)	Return instruction	**TOK_ESCAPE**
Unwind token	Non-local jump (longjmp)	**StateIndicator::escape()**
Clear-State-Indicator token (**)SIC**)	Reset / stack flush	**class Cmd_SI**
Error token	Trap / interrupt	**TOK_ERROR** / throw **Error**
Nil token	No-op / fall-through	**TOK_NO_VALUE** / **TOK_VOID**

ISO 13751 concept	CPU / VM analogue	GNU APL
Defined function	Subroutine	**`class UserFunction`** (**`src/UserFunction.hh`**)
Calling a defined function	CALL instruction (push frame)	UserFunction::eval_XXX() (creates new **`StateIndicator`** entry)
Returning from a defined function	RETURN instruction (pop frame)	Workspace::pop_SI()
Attention flag	Hardware interrupt flag	**`Workspace::interrupt_is_raised()`**

That hypothetical machine has a striking number of parallels with a silicon CPU. Table 1-1 above maps **ISO 13751** concepts to their CPU or virtual-machine analogues and to their concrete counterparts in GNU APL.

The fetch–decode–execute cycle is most visible in the **`Defined-Function-Control`** evaluation sequence (§13.4.2), which the standard writes as:

```
Set I to one.
Repeat:
  Set Z to evaluate-line of current-function-line I.
  If Z is branch, set I to the content of Z.
  If Z is value or nil, set I to I plus one.
  If I is not in 1..last-line, return result.
```

This is word-for-word the same structure as a simple interpreter loop: fetch the instruction at the program counter, decode it (evaluate-line), check the result class (branch vs. fall-through), and update the PC.

The analogy is not accidental. APL's originators designed the language around a notation that could be *executed* directly, without compilation; the ISO standard formalised that design as an explicit machine model. GNU APL's **`Prefix.cc`** is the direct implementation of that machine. Its **`eval_()`** function is the fetch–decode–execute loop, **`StateIndicator`** is the call stack, and every **`eval_XXX()`** return value is an opcode result — either a value Token or an exception Token — that the loop dispatches on.

1.2.2. The Interpreter at a Glance

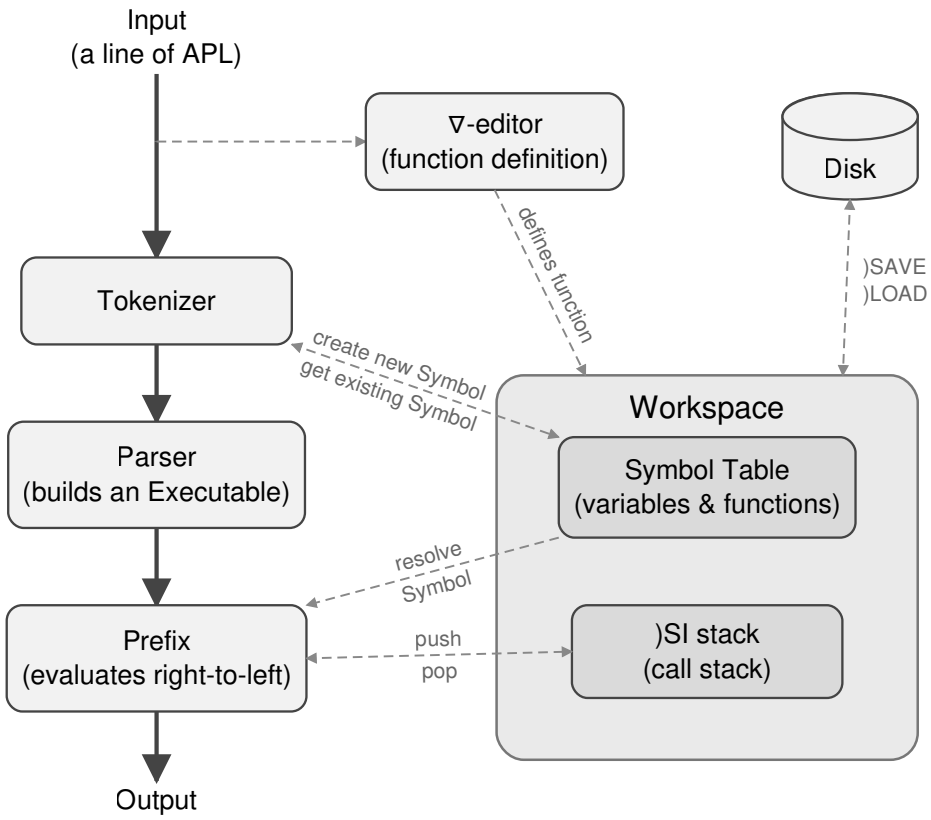


Figure 1. The interpreter's top-level pipeline

Figure 1 above puts Table 1-1 from earlier into motion: the path a single line of input takes from the moment it is typed to the moment a result comes back.

- **Tokenizer** turns raw characters into a sequence of **`Token`** objects — numbers, names, glyphs, strings. Every name token it produces is looked up in the Symbol Table in both directions: an existing variable or function if one is already defined under that name, or a new, as-yet-unbound symbol-table entry if not.
- **Parser** assembles that token sequence into an **`Executable`**, one of three kinds: a **`StatementList`** for something typed directly at the session prompt, a **`UserFunction`** body for a defined function, or an **`ExecuteList`** for the character-string argument of monadic **⍎** (Execute). An **`ExecuteList`** is compiled and run as a one-off statement with its own transient state-indicator frame, distinct from both of the other two. A **`UserFunction`** is also entered into the Symbol Table under its own name, the way any variable would be. A **`StatementList`** or **`ExecuteList`** is never a Symbol Table entry — each exists only as a)SI stack frame for the duration of its own evaluation, and is removed automatically once that finishes.

- **Prefix** is the evaluator proper — the fetch-decode-execute loop from Table 1-1 above, scanning right-to-left and reducing the statement phrase by phrase.
- **Workspace** is not itself a single piece of state so much as the container for two things Prefix depends on constantly. One is the **Symbol Table**, holding every variable and function currently defined, which Prefix reads from whenever a name is resolved. The other is the **)SI stack**, the call stack, which Prefix pushes to and pops from whenever a function calls another function, one entry per suspended or active invocation. The Workspace as a whole is also what **`)SAVE`** writes out to disk and **`)LOAD`** reads back in — the unit of persistence is the whole container, not its individual parts.

Later chapters take each of these apart in turn — Chapter 7 covers the Tokenizer, Chapter 8 the Parser, Chapter 9 the State Indicator. But this shape is worth holding onto as a map before the terrain gets detailed: input flowing through Tokenizer, Parser, and Prefix, while both consult the Workspace's Symbol Table and)SI stack along the way.

1.3. The GNU Project and GNU APL

The GNU Project, launched by Richard Stallman in 1983 as part of the Free Software Foundation, aims to provide a complete free (as in freedom) operating system and toolchain. GNU APL is a component of that project: a free, standards-conforming implementation of **ISO/IEC 13751**, hosted on the GNU Savannah infrastructure and distributed under the GNU General Public License.

The personal motivation for GNU APL reaches back to DATIS-P (described in detail in Chapter 21). When the 32-processor prototype was running, the plan was to port a commercial APL interpreter — *APL68000*, produced by MicroAPL of London — to run on the DATIS-P master processor. The prerequisite work had been done: the Mirage operating system had already been ported to the master. The hardware itself worked, and the parallel algorithms were proven.

The port never happened. The students who had built the DATIS-P prototype completed their Master's theses and moved into industry positions. At roughly the same time the author finished his Ph.D. and his contract as a university assistant came to an end. DATIS-P continued to be used for other parallel applications, but never ran a full APL interpreter.

GNU APL began years later, around 2008. The trigger was almost incidental: the author was experimenting with Unicode and noticed that every APL glyph already had a code point. A standards-conforming APL no longer needed a special font or a custom keyboard driver to exist — just an interpreter. The project became, in part, an attempt to finish the job DATIS-P had left undone: to build the APL interpreter that DATIS-P was always meant to have. GNU APL has

never run on DATIS-P hardware (the prototype is long gone), but the parallel execution framework it carries is a direct software re-implementation of the same ideas. In that sense, the project is complete.

1.4. What Makes APL Unusual

One of the remarkable features of early APL was how accessible it proved to people who had no background in programming. This was not an accident. Iverson had designed the notation as a tool for thought, not as a specification language for machines. The array orientation meant that a trained accountant or scientist could often express a calculation in a single line that would have required a full subroutine in FORTRAN or COBOL. The interactive, conversational nature of APL interpreters of the 1970s reinforced this: you typed an expression, pressed Enter, and the answer appeared. There was no compile step, no link step, no submission to a batch queue. The feedback loop was immediate.

The author encountered this phenomenon at first hand when he was employed part-time at a small tax-advisory office — a firm that prepared tax declarations for its clients. The owner of the firm was an economist by education, not a programmer in any formal sense. Nevertheless he had acquired an IBM 5110 — a self-contained desktop computer marketed by IBM in the late 1970s with a built-in APL interpreter. And he had written a collection of APL workspaces to automate the bookkeeping tasks that occupied most of his staff’s time: posting incoming invoices to accounts, reconciling ledger columns, summarising quarterly figures. The workspaces were modest by any technical standard, but they worked reliably, and the economist maintained them himself. He never thought of himself as a programmer.

In due course the firm outgrew the IBM 5110. The owner’s next system was APL68000, an interpreter for Motorola 68000-based hardware produced by MicroAPL of London, running under the Mirage operating system. Mirage was a multi-user environment that could support up to five simultaneous APL sessions, each connected through a dedicated terminal. The bookkeeping workspaces were ported without difficulty — APL’s portability across implementations was already a practical reality — and several members of staff could now work in APL concurrently.

At the same period the author was employed as a system programmer for Mirage, in parallel with his studies of mathematics at the university. That combination of duties was unusual but productive: it meant that when the DATIS-P project later needed an operating system for its master processor, Mirage was the natural choice. The author knew it thoroughly, had already ported it to related hardware, and could predict where the difficulties would lie. The tax office and the parallel machine were separated by years and by context, but they were connected by a thread: Mirage.

1.5. Notes

The primary sources behind this chapter are short and worth reading directly rather than at second hand. Iverson's 1962 book, *A Programming Language*, is where the notation this whole codebase ultimately implements was first written down — as mathematical notation, not as a specification for a machine. APL's identity as "a tool for thought" rather than "a way to talk to a computer" is Iverson's own framing. He argued it at length eighteen years later, in his 1979 Turing Award lecture, *Notation as a Tool of Thought* (*Communications of the ACM* 23(8), 1980).

The tax-office anecdote earlier in this chapter is not a digression from that thesis; it is closer to a field test of it. Iverson's argument was that a well-designed notation could make a calculation easier to **think**, independent of any intention to execute it on a machine. And an economist who never thought of himself as a programmer — automating his own bookkeeping on an IBM 5110 because the notation was tractable enough to pick up without formal training — is closer to Iverson's stated goal than most of what gets called "user-friendly" programming today.

GNU APL's own place in that lineage is not incidental: it exists because **ISO/IEC 13751** gives *APL, Extended* a written specification precise enough to implement independently of any one vendor's interpreter (this chapter's **`The ISO Standard`** section). That is the same property — precision independent of any particular machine — that made Iverson's 1962 notation usable to describe algorithms years before an interpreter for it existed at all.

1.6. Exercises

1. [1] Download and build GNU APL from the Savannah repository following the instructions in Chapter 2. Confirm that **`src/apl`** starts and accepts input.
2. [2] Find the **ISO/IEC 13751** standard and identify three features it specifies that are not present in Iverson's original 1962 notation.
3. [2] Read Iverson's 1991 retrospective essay *A Personal View of APL* (*IBM Systems Journal* 30(4); see the Bibliography). Summarise in one page which design decisions he would make differently in hindsight.

2. Building and Developing

This chapter explains how to obtain the source, build the interpreter, and work with the code day-to-day. It also introduces the definition-file mechanism (§2.6), which appears throughout the source and is easiest to explain here, before the reader encounters it in later chapters.

2.1. Prerequisites

The only hard requirement is a C++ compiler and GNU Make. Any version of **`g++`** that supports C++11 or later will do; GCC 7 and above have been used successfully. On Debian-based systems the essentials are:

```
apt-get install g++ make
```

Autotools (**`autoconf`**, **`automake`**, **`libtool`**) are needed only if **`configure.ac`** or a **`Makefile.am`** is modified; a fresh checkout already contains a generated **`configure`** script, so ordinary users never need them.

Several optional libraries enable additional system functions. GNU APL compiles and runs without any of them; a call to a function whose library is absent raises **`SYNTAX ERROR`** instead of executing. The **`)MORE`** command, issued immediately after such an error, names the missing library and suggests an **`apt install`** command.

Table 3. Optional libraries and the functions they enable

Library	Package (Debian)	Enables
libsqlite3	**`libsqlite3-dev`**	**`SQL`** (SQLite interface)
libpq	**`libpq-dev`**	**`SQL`** (PostgreSQL interface)
libfftw3	**`libfftw3-dev`**	**`FFT`** (Fast Fourier Transform)
libgsl	**`libgsl-dev`**	**`MX`** (matrix operations via GSL)
gtk3	**`libgtk-3-dev`**	**`GTK`** widget interface and **`PLOT`**
libxcb	**`libxcb1-dev`**	**`PLOT`** via XCB (alternative to GTK)

2.2. Obtaining the Source

The authoritative repository is hosted on Savannah under the GNU project. Check out the development trunk with Subversion:

```
svn checkout svn://svn.savannah.gnu.org/apl/trunk gnu-apl
```

Releases are also published as tarballs on **`https://ftp.gnu.org/gnu/apl/`**.

GNU APL is **not** hosted on GitHub. Some GitHub users have cloned GNU APL into private subdirectories of their own accounts, but those clones are not affiliated with the GNU APL project and may be outdated or modified. Savannah itself provides a combined SVN/Git repository, so the same trunk can also be cloned with Git directly from Savannah:

```
git clone https://git.savannah.gnu.org/git/apl.git gnu-apl
```

For active development SVN is the authoritative source; use the Savannah Git mirror only if SVN is unavailable or inconvenient.

The repository history extends back to approximately 2008 and contains the full evolution of the interpreter.

2.3. The Autotools Build System

Before a single line of interpreter code is written, a project has to choose its build machinery. This is one of the first real design decisions. It is not really a decision for the maintainer at all — it is a decision about what every future **user** will need on their own machine just to turn a source tree into a running **`apl`**. GNU APL considered both Autotools and CMake and chose Autotools, and the deciding factor was exactly that user-facing cost. A CMake-based build requires CMake itself to be installed first — an extra tool dependency on top of a compiler. An Autotools-based build, by contrast, ships its generated **`configure`** script and **`Makefile.in`** templates already checked into the source tree (see below). So building from a checkout needs nothing more than a POSIX shell, **`make`**, and a C++ compiler — tools already present on essentially any GNU/Linux, macOS, or BSD system. **`./configure && make`** with no separate build-tool installation first was the goal, and it is why Autotools, not CMake, underlies everything in this chapter.

GNU APL uses the standard GNU Autotools chain. The two handwritten input files are **`configure.ac`** (project-level settings and feature tests) and **`src/Makefile.am`** (the list of source files and build rules). Running **`autoreconf`** processes these through **`autoconf`** and **`automake`** to produce the **`configure`** shell script and the **`Makefile.in`** templates. Both generated files are checked into SVN, so contributors who only edit **`.cc`** and **`.hh`** files never need to run **`autoreconf`**.

Figure 2 below shows the whole chain. The single **`configure.ac`** and the handwritten **`Makefile.am`** in each directory are run through **`autoreconf`** once (by a maintainer, only when one of them changes) into **`configure`** and a generated **`Makefile.in`** per directory, both shipped with the source. **`./configure`** then turns those into the concrete files **`make`** actually reads.

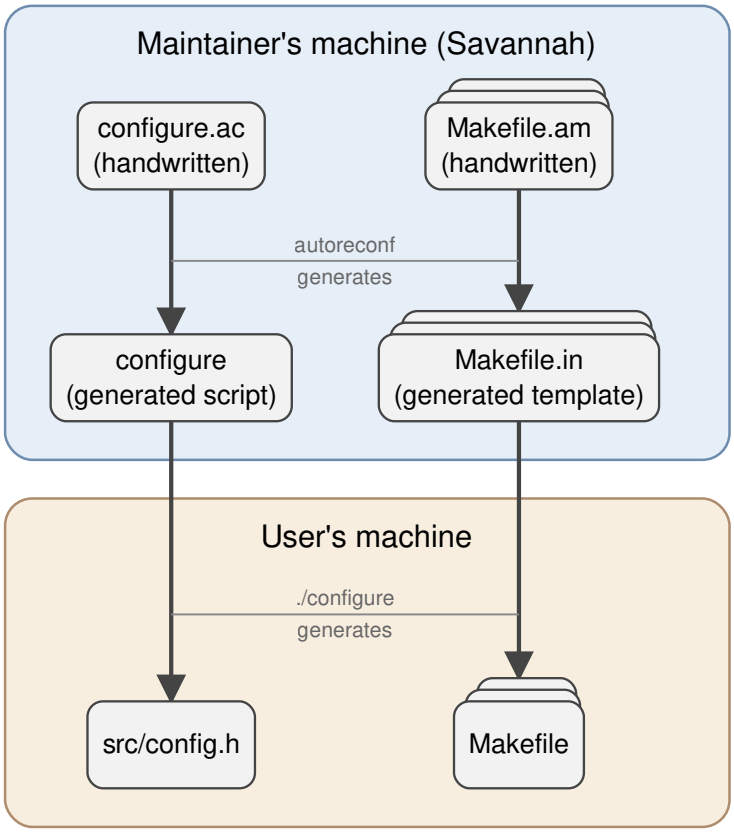


Figure 2. The Autotools build flow

The normal build sequence is:

```
./configure
make -C src
sudo make install
```

In practice, developers working directly on the source usually skip the last step: **`src/apl`** already runs from the build tree, and installing a fresh binary system-wide after every change is rarely worth the extra step. **`sudo make install`** matters when GNU APL is being set up as an ordinary installed tool for others to run, not while iterating on it.

./configure probes the system for available libraries and compiler features, then writes **config.h** and the concrete **Makefile** files. **config.h** is a single file, **src/config.h**. It holds the entire body of machine-specific information relevant to the sources in and below **src/** — which libraries were found, which compiler features are available, and so on — as a set of preprocessor **#define**-s. That is why it is **#include**-d, directly or indirectly, in a large number of the project's **.hh** and **.cc** files. It is the one place where "does this build have GSL?" or "does this platform have **sem_timedwait()**?" gets answered, once, at configure time, instead of every translation unit probing for itself. **make -C src** compiles only the **src/** subdirectory, which is where the interpreter lives; building the full tree is not necessary for normal development. The resulting binary is **src/apl**.

make tracks header and source dependencies automatically. After editing a **.cc** file, re-running **make -C src** recompiles only the changed translation unit and relinks. A full rebuild from a clean tree takes several minutes on typical hardware; incremental builds after a small change take seconds.

sudo make install installs **apl** to **/usr/local/bin** and the workspace libraries to **/usr/local/lib/apl/**. The installation prefix defaults to **/usr/local** and can be changed at configure time:

```
./configure --prefix=/opt/apl
```

2.4. Key Configure Options

The full list of GNU APL-specific options is documented in **README-2-configure**. The most important ones for a developer are:

--with-sqlite3
: Enable **SQL** backed by SQLite.

--with-postgresql
: Enable **SQL** backed by PostgreSQL.

--with-libapl
: Build GNU APL as a shared library (**libapl.so**) rather than an executable. Used for embedding the interpreter in other programs. The library must be initialised with **init_libapl()** before any other call.

ASSERT_LEVEL_WANTED=*n*
: Controls how many internal consistency checks are compiled in. The default is **1** (light checks only). Set to **2** when tracking down a bug: all **Assert()** and **Assert1()** macros are active and abort with a diagnostic if violated. Set to **0** for maximum performance.

SECURITY_LEVEL_WANTED=*n*
: Restricts interpreter capabilities for use in untrusted environments such as CGI scripts. The default **0** enables everything.

CORE_COUNT_WANTED=*n*
: Selects the number of CPU cores used by the parallel execution path. The default **0** uses sequential code. See **README-8-parallel** for details.

Options are combined freely on the same command line:

```
./configure --with-sqlite3 ASSERT_LEVEL_WANTED=2
```

Not every setting is a **./configure** argument. Ordinary Autotools convention also honours compiler and linker variables (**CXX**, **CXXFLAGS**, **LDFLAGS**, and similar) set to the **left** of the **./configure** invocation itself, rather than passed as arguments to it. **build/build_all**, the project's own multi-configuration nightly-build driver, uses exactly this form for its AddressSanitizer configuration:

```
CXXFLAGS="-fsanitize=address -fno-omit-frame-pointer -g -O1" \
LDFLAGS="-fsanitize=address" \
   ../configure ASSERT_LEVEL_WANTED=2
```

2.5. Out-of-Tree Builds

By default, **make -C src** places object files alongside the source files in **src/**. If **./configure** is re-run with different options, the new object files overwrite the old ones, making it impossible to keep two differently configured builds side by side.

Out-of-tree (VPATH) builds solve this by redirecting all output to a separate directory:

```
mkdir build-debug
cd build-debug
../configure ASSERT_LEVEL_WANTED=2
make
```

Every object file and the final **apl** binary land in **build-debug/src/**, leaving the source tree untouched. Multiple build directories can coexist; a common setup is one directory for a release build and one with full assertions enabled for testing.

2.6. Definition Files

Before diving into any particular subsystem, it is worth understanding one coding technique that appears throughout the GNU APL source: the *definition file*. Roughly twenty **.def** files live in **src/**, and every major subsystem uses at least one. Reading any significant piece of GNU APL source without understanding this mechanism will leave the reader puzzled by **#include** directives that appear inside **switch** statements or enum bodies.

2.6.1. What Are Definition Files?

A definition file is a plain text file — not a normal header — that contains nothing but a sequence of invocations of a single macro. By convention, definition files always carry the **`.def`** extension, which immediately distinguishes them from ordinary headers (**`.hh`**) and implementation files (**`.cc`**). The file does *not* define that macro itself; it only calls it. The caller supplies the definition:

```
#define err_def(name, text, major, minor)  /* expand however needed */
#include "Error.def"    // calls err_def(...) for every error code
#undef  err_def
```

Because the macro is defined by the caller, the same **`.def`** file can be **`#include`**-d any number of times in the same translation unit, each time with a completely different expansion. The **`.def`** file is the data; the macro definition is the operation applied to that data.

2.6.2. Purpose

Many data structures in an interpreter are fundamentally *relations* between two or more properties of the same item. Error codes and their human-readable texts are one example; token tag values and their printable names are another; logging flags and their labels a third.

The traditional C++ solutions both have the same weakness. The first is a hand-written **`switch`**:

```
const char *
error_text(ErrorCode ec)
{
   switch (ec)
      {
        case E_NO_ERROR:    return "No Error";
        case E_DEFN_ERROR:  return "DEFN ERROR";
        // ...
      }
}
```

The second is a vector or array of structs:

```
struct ErrorEntry { ErrorCode code; const char * text; };
static const ErrorEntry error_table[] =
{
   { E_NO_ERROR,   "No Error"    },
   { E_DEFN_ERROR, "DEFN ERROR"  },
   // ...
};
```

Both approaches share a critical maintenance hazard: the enum declaration lives in one file and the table or switch lives in another. When a new error code is added to the enum, nothing in the compiler reminds the programmer to add the corresponding entry to the table.

The omission silently produces a wrong result — a missing **`case`** is simply skipped, and an incomplete table returns a null pointer or a garbage entry. In a large codebase with dozens of codes maintained over years by different people, such omissions are routine.

Definition files eliminate the hazard by making the list of items the *only* thing that needs to change. All derived artifacts — the enum, the switch, the table — are generated from the same source by including it with different macro definitions. Adding a new item in one place automatically updates every consumer.

2.6.3. A Motivating Example: Error Codes

`Error.def` records every error that GNU APL can raise. Each row carries the symbolic name, the display text, and a two-part numeric code:

```
// Error.def (excerpt)
err_def(NO_ERROR,    "No Error",      0,      0 )
err_def(DEFN_ERROR,  "DEFN ERROR",    0, 0x0B01 )
err_def(RANK_ERROR,  "RANK ERROR",    2,      1 )
```

This single file is included three times, each time with a different macro definition.

`ErrorCode.hh` generates the enum:

```
enum ErrorCode
{
#define err_def(name, text, major, minor) \
        E_ ## name = (major) << 16 | (minor),
#include "Error.def"
#undef  err_def
};
```

`Error.cc` includes it a second time to build the text-lookup function:

```
const char *
Error::error_name(ErrorCode ec)
{
   switch (ec)
      {
#define err_def(name, text, major, minor) \
        case E_ ## name: return text;
#include "Error.def"
#undef  err_def
        default: return "unknown error";
      }
}
```

And a third time for the numeric-code lookup, the severity test, and so on. Adding a new error — say **`MEMORY_FULL`** — requires exactly one new line in **`Error.def`**. Every derived function is updated automatically at the next build; there is nothing else to touch and nothing to forget.

2.6.4. Usage Across the Codebase

The same **`err_def`**-style X-macro pattern recurs throughout **`src/`**, one **`.def`** file per subsystem. Table 2-2 below catalogues the ones a maintainer runs into most often.

Table 4. The most frequently encountered `.def` files

File	Macro	What each row defines
`Error.def`	`err_def`	An APL error: symbolic name, display text, major and minor numeric code. Used to generate the `ErrorCode` enum, text-lookup functions, and severity tests.
`Logging.def`	`log_def`	A logging flag: symbolic name, initial on/off value, and a label string shown by the `]LOG` command.
`Id.def`	`pp`	An APL primitive identifier: tag name, glyph string, and enum value. Drives the `ID` enum that labels every built-in function and operator.
`Token.def`	`TD`	A token tag: tag name, token class, value type, and associated ID. Generates the `TokenTag` enum and the token-class lookup table.
`SystemVariable.def`	`ro_sv_def` / `rw_sv_def`	A system quad-variable: implementing class, short name (e.g. `"IO"`), and help text. Two macro variants distinguish read-only from read-write variables.
`Command.def`	`cmd_def`	A workspace command: name (e.g. `")LOAD"`), handler function, and help text. Used to build the command-dispatch table.
`Avec.def`	`char_def`	One entry in the APL character set (⎕AV): Unicode code point, printable representation, and token tag.
`Macro.def`	`mac_def`	A built-in APL macro: name and source text. Macros are small APL defined functions compiled into the interpreter at build time.
`Performance.def`	`perfo_1` / `perfo_2`	A performance counter: function name, argument signature, display string, and baseline cycle count used by the `]PERF` command.

File	Macro	What each row defines
Quad_MX.def	**mx_def**	A □**MX** (matrix extension) subfunction: name and axis number.

The **src/** directory contains roughly twenty **.def** files. A reader who spots an unfamiliar **#include** of a **.def** file in the middle of a function or class body should look for the immediately preceding **#define** of its macro. That definition reveals what the inclusion is producing at that point. Table 2-2 above lists the most frequently encountered definition files.

The pattern for extending any of these subsystems is uniform: add one row to the appropriate **.def** file, rebuild, and every consumer is updated automatically.

2.7. The Development Cycle

The inner loop for working on the interpreter is short:

```
# 1. edit one or more .cc / .hh files in src/
# 2. rebuild
make -C src
# 3. run a testcase
src/apl --id 1010 -T src/testcases/Foo.tc
```

The **--id 1010** flag assigns a workspace identifier. This is required when multiple APL processes run concurrently (for example during **make test**) because APserver uses the ID to route shared-variable messages; using a unique ID for an interactive session avoids collisions.

To stop on the first mismatch rather than running to the end:

```
src/apl --id 1010 --TM 3 -T src/testcases/Foo.tc
```

The **--TM** flag sets the test-mismatch behaviour: **0** continues silently, **1** stops on error, **3** stops on the first output mismatch. The latter is the most useful during active debugging because it leaves the interpreter sitting at the failing expression.

The available test targets differ in scope and speed:

```
make -C src test3     # normal test run — fast, used routinely
make -C src test4     # includes invariant tests (□RVAL-based)
make -C src test      # complete suite — slow, includes □SVx tests
make -C src test1     # complete suite, stop on first failure
```

test3 is the target used for day-to-day development. **test4** additionally runs the invariant testcases (see Chapter 3). **test** runs the full suite including testcases for the shared-variable functions **⎕SVO**, **⎕SVC**, **⎕SVR**, and **⎕SVQ**. These are provided for backward compatibility with IBM APL2 and older APL1 interpreters and are otherwise considered outdated; their tests involve the APserver daemon and take considerably longer than the rest of the suite. For routine work, **test3** or **test4** is sufficient.

./configure does not need to be re-run between edits unless a build option changes or a new source file is added to **Makefile.am**.

2.8. Notes

GNU APL was started around 2008 and has always used Autotools as its build system. At that time Autotools was the established portable build infrastructure for GNU projects. It remains the right choice for a codebase that must compile cleanly on GNU/Linux, macOS, and various BSD systems without imposing tooling requirements beyond a C++ compiler and **make**.

The decision to keep object files in **src/** rather than a separate build directory is a deliberate simplicity choice for the common case: most users build exactly one configuration. Developers who need multiple configurations simultaneously use VPATH builds (§2.5), which the Autotools infrastructure supports without any special effort.

2.9. Exercises

1. [1] Configure and build GNU APL in an out-of-tree directory. Confirm that **src/apl** and the out-of-tree **apl** produce identical version strings.
2. [2] Run **./configure --help** and identify all options that control which optional quad functions are compiled in. List them and the library each requires.
3. [2] Build GNU APL twice in two separate out-of-tree directories: once with **ASSERT_LEVEL_WANTED=0** and once with **ASSERT_LEVEL_WANTED=2**. Compare the sizes of the two resulting **apl** binaries and measure the difference in runtime on a simple benchmark such as **+/⍳1000000**.

3. The Test Infrastructure

A working interpreter is not just correct code — it is code that can be *proven* correct after every change. GNU APL ships with roughly 270 testcase files covering the entire APL standard and most of the interpreter's extension functions. Running them takes about a minute and costs nothing; breaking them silently costs much more. This chapter explains how the test infrastructure works, how to read and write testcase files, and how to handle the inevitable occasion when a deliberate change makes the existing expected output wrong.

3.1. Test File Format

A testcase file is a plain text file with the extension **`.tc`**. It contains a sequence of APL expressions interleaved with their expected output. The interpreter reads the file top to bottom, executes each input line, compares the actual output with the expected output, and records any discrepancy.

3.1.1. A Minimal Example

The simplest possible testcase file looks like this:

```
⍝ Add.tc
⍝ ═══════

      5+1 2 3
6 7 8
      1 2 3+4 5 6
5 7 9
```

3.1.2. How Lines Are Classified

It would be natural to assume that the interpreter classifies lines by their content — that an APL expression is an input line and anything else is expected output. That assumption is wrong.

The testcase file is a single sequential byte stream that is shared by two readers running at the same time. There is no pre-scan; no line is labelled before execution starts.

Reader 1 — the APL input reader (**`IO_Files::get_file_line`**) feeds lines to the interpreter one at a time, exactly as an interactive user would type them. Every line it delivers is logged with a **`---->`** prefix.

Reader 2 — the output comparator (`DiffOut::overflow`) intercepts every character that the interpreter writes to standard output. When it receives a newline — the end of one output line — it immediately calls `IO_Files::read_file_line` to pull the next line from the file and uses it as the reference for comparison. That pulled line is the *expected output line*; it is consumed silently and never fed to the interpreter.

The two readers advance through the same tape in strict lock-step:

1. Reader 1 delivers a line to the interpreter.
2. The interpreter executes it and produces zero or more output lines.
3. For each output line, Reader 2 pulls the next line from the tape and compares it.
4. Reader 1 then delivers the next line to the interpreter, and so on.

The consequence is that *a line is classified solely by its position in the file, not by its content*. A blank line, a comment, or even an APL expression could all appear in either role:

- When the interpreter has nothing left to output after the previous input line, Reader 1 takes the next line as input. Comment lines (`⍝`) and blank lines fall into this category because they produce no output.
- When the interpreter has just finished printing an output line, Reader 2 takes the next line as the reference — whatever that line happens to contain.

This also explains how multiline output works: an expression that prints three lines causes Reader 2 to consume the next three lines from the file before Reader 1 gets another turn.

3.1.3. What Triggers Each Reader

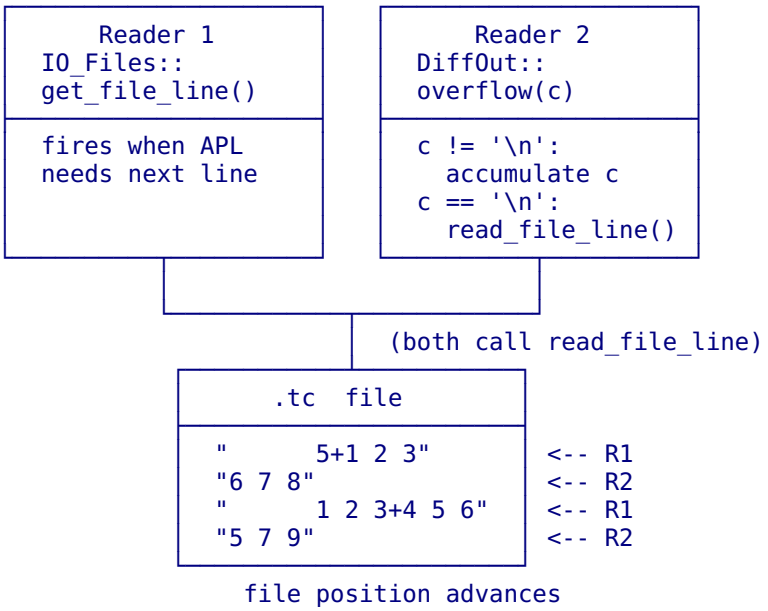
No explicit controller decides whose turn it is. The two readers are activated by two independent events:

- Reader 1 fires when the interpreter's input loop calls `LineInput::get_line()` to request the next APL expression. This call is made only after any output from the previous expression has been fully flushed, so Reader 2 has already consumed every expected output line before Reader 1 advances the tape again.
- Reader 2 fires inside `DiffOut::overflow()` — the virtual `overflow` method of a `std::filebuf` subclass that `cout` has been redirected to. It is called for every character APL writes. At the top of `overflow()`, `InputFile::is_validating()` is checked; it returns `true` only when the current input file is a `.tc` testcase (i.e., `InputFile::test == true`). For ordinary scripts that check fails and the character is forwarded unchanged — no comparison

happens. For testcase files, characters that are not newlines are accumulated in a buffer. When a newline arrives, the accumulated line is complete and **`read_file_line()`** is called to pull the next reference line from the tape for comparison.

Both readers call the same **`IO_Files::read_file_line()`**, which advances the single shared file position, as Figure 3 shows.

Figure 3. Two independent readers sharing one file position



The interleaving is entirely determined by the control flow: after the interpreter has finished producing output for one expression and returns to its input loop, Reader 1 is next; mid-output, every **`'\n'`** triggers Reader 2. There is no mode variable that toggles between readers; they fire independently, gated only by **`InputFile::is_validating()`**, which returns **`true`** whenever a **`.tc`** file is the current input.

The indentation of input lines is conventional (six spaces in most **`.tc`** files) but carries no syntactic weight; the interpreter strips leading whitespace.

The file does not need a header or a terminator. When the file ends, the test runner prints a summary and closes.

3.1.4. System Commands and Defined Functions

APL system commands (**`)CLEAR`**, **`)LOAD`**, **`)WSID`**, and so on) and defined function bodies introduced with **`∇`** are also valid input. The interpreter treats them exactly like ordinary APL expressions: they are executed and any output they produce is compared line by line with the expected output in the file.

A defined function body in a **`.tc`** file must be entered as the interpreter would display it — one line per statement, terminated by a closing **`∇`**:

```
∇Z←A FUN1 B
 Z←'result'
∇
```

3.1.5. The Log File

Every testcase run produces a **`.tc.log`** file alongside the **`.tc`** file. This log is the primary diagnostic tool: it records every input line, every line of expected output, and — most importantly — every mismatch.

A matching line is shown with the **`==`** prefix:

```
----> 5+1 2 3
== 6 7 8
```

A mismatch shows three lines: the actual APL output, the reference (expected) output, and a delta line marking the first position where they differ:

```
----> ⎕TS
apl: ···2026 6 24 13 45 12 317···
ref: ···2026 6 24 13 45 12 000···
 Δ : ···                   ^
```

Here the millisecond field differs — the reference contains **`000`** but APL produced **`317`**. Without some form of matching, the timestamp would fail on every run. The **`⁰`** wildcard (§3.4) solves this: writing **`2026 6 24 13 45 12 ⁰`** as the reference line matches any digit sequence in that position. The log then shows **`== 2026 6 24 13 45 12 317`** instead of a mismatch.

3.2. Running the Test Suite

3.2.1. Make Targets

The **`src/Makefile.am`** defines four test targets, all invoked from the **`src/`** directory:

```
make -C src test3   # normal development run
make -C src test4   # includes invariant testcases (⎕RVAL-based)
make -C src test    # complete suite including ⎕SVO/⎕SVC/⎕SVR/⎕SVQ tests
make -C src test1   # complete suite, stop on first failure
```

test3 is the target to use for routine development. It runs all testcases in **src/testcases_3/** (which is a directory of symbolic links into **src/testcases/**; see §3.6) and takes about a minute. **test** additionally runs the shared-variable testcases, which involve the APserver daemon and are significantly slower. They are provided for compatibility verification and are not needed for most development tasks.

3.2.2. Running a Single File

During active debugging it is often useful to run just one testcase in isolation:

```
src/apl --id 1010 -T src/testcases/Foo.tc
```

The **--id 1010** flag assigns a workspace identifier. This is required because multiple simultaneously running interpreter processes — as during **make test** — each need a unique ID so that the APserver daemon can route shared-variable messages to the right process. Using ID 1010 for a single interactive session avoids collisions with background processes.

The **-T** flag (capital T) names one or more testcase files: it consumes every command-line argument that follows, up to the next recognized flag or a literal **--**, and runs them all in a single interpreter session. It does **not** do any wildcard expansion of its own, so a shell glob must be left unquoted:

```
src/apl --id 1010 -T src/testcases/*.tc
```

Here the shell expands **src/testcases/*.tc** into a separate argument for every matching file **before** apl ever sees the command line, and that whole list is consumed by the single **-T** command line option. Because it is one interpreter invocation, **summary.log** is updated once with the combined results.

3.2.3. Controlling Mismatch Behaviour

The **--TM** flag sets the test-mismatch behaviour:

```
src/apl --id 1010 --TM 0 -T src/testcases/Foo.tc   # continue silently
src/apl --id 1010 --TM 1 -T src/testcases/Foo.tc   # stop on APL error
src/apl --id 1010 --TM 3 -T src/testcases/Foo.tc   # stop on first mismatch
```

Mode 3 is the most useful during debugging: the interpreter halts at the exact expression that produced unexpected output and drops into interactive mode, allowing the programmer to inspect variables, run **)SI**, and experiment before continuing.

3.3. The Summary Log

After processing every testcase file, the interpreter appends a line to **`src/testcases/summary.log`**. The file is created on the first run and extended on subsequent runs. Each line records the error count, wall-clock duration, and file name:

```
Errors   Time File
========================
  0   0.139 s  Add.tc
  0   0.051 s  APnnn.tc
  1   0.203 s  Quad_PLOT.tc (0 APL, 0 assert, 1 diff, 0 parse)
  0   0.087 s  ZZ36_Regression.tc
```

A non-zero error count immediately identifies the failing file. The parenthesised breakdown after the count splits it into four categories: **`APL`** (testcase-mechanism errors, see **`]EXPECT`** below), **`assert`** (failed **`⎕ASSERT`** or internal C++ assertions), **`diff`** (actual output did not match expected output — the category an ordinary wrong **`DOMAIN ERROR`** or **`RANK ERROR`** falls into), and **`parse`** (the testcase file itself was malformed). Each category needs a different diagnostic approach.

The **`make test3`** target passes all testcase files to a single interpreter invocation via a glob, so **`summary.log`** is written once with the complete results. An older workflow that called the interpreter once per file in a shell loop would have each invocation overwrite **`summary.log`**, leaving only the last file's results — a subtle trap that the single-invocation approach avoids.

3.4. Wildcards in Expected Output

Many APL expressions produce output that varies legitimately between runs or between platforms. **`⎕TS`** returns the current timestamp; **`⎕WA`** returns available workspace memory; a pointer printed for debugging purposes changes between processes. If expected output contained exact values for these, every run would report a false failure.

The test infrastructure addresses this with a set of *wildcard characters* embedded in the expected output. They are Unicode superscript digits and the letter ⁿ. The matching logic lives in **`DiffOut::different()`** in **`src/DiffOut.cc`**.

Table 5. Wildcard characters recognized by `DiffOut::different()`

Wildcard	Matches	Typical use
`⁰`	One or more decimal digits	Numeric counts, timestamps, IDs: **`1010 ⁰ ⁰ ⁰ ⁰`** matches **`⎕AI`** output.
`¹`	Zero or more space characters	Padding in formatted output that varies by value width.
`²`	A floating-point number (digits, optional overbar `¯`, `.`, `E`, `J`)	Any real or complex scalar whose exact value is platform-dependent.
`³`	The rest of the line (anything)	File paths, workspace names, or any suffix whose value does not matter.
`⁴`	An optional overbar `¯`	Sign of a result that may be positive or negative depending on the platform.
`⁵`	ASCII **`+`** or **`-`**	Signed exponents in formatted output.
`⁶`	**`0`** or **`1`** depending on whether rational numbers are compiled in	The result of **`28 ⎕CR 42`** which is build-configuration dependent.
`ⁿ`	An optional SI multiplier: **`m`**, **`n`**, **`u`**, or **`µ`**	Performance counter output that scales with the magnitude.

A wildcard matches *greedily*: `⁰` consumes all consecutive digits starting at the current position in the APL output. The rest of the expected line after the wildcard must then match the rest of the APL output literally.

Here is a real example from **`APnnn.tc`**:

```
      ⎕AI
1010 ⁰ ⁰ ⁰ ⁰
```

`⎕AI` returns a five-element vector: workspace ID, connect time, CPU time, tokens, and keying time. The ID is always 1010 when **`--id 1010`** is used; the four timing values change every run. The four `⁰` wildcards match whatever digits appear in those positions.

accept_tc.py (§3.7) does not invent wildcards for output that never had one, but it does not discard existing ones either: for each line it regenerates, it first checks whether the **previous** expected line — the one the failing run actually diffed against — still matches the new actual output once its wildcards are taken into account (the same **DiffOut::different()** logic the interpreter itself uses, ported to Python). If it still matches, the old, wildcarded line is kept verbatim instead of being overwritten with a literal value. Only a line whose previous expected value no longer matches at all — an actual regression, not a wildcarded field taking on a new value — is replaced with the literal new output, exactly as before. This works because a wildcard, by construction, never causes a genuine diff failure on its own: the two failure modes are disjoint, so checking "does the old wildcarded line still match" is a safe, conservative test for "is this the same output that just happens not to have changed."

3.5. Conditional Skipping:]NEXTFILE

Some testcases are only meaningful for certain build configurations. A testcase for **□SQL** makes no sense if SQLite was not compiled in; a testcase for **□PLOT** in GTK mode cannot run if GTK is absent. Rather than making a separate testcase directory for every configuration, the test infrastructure provides the **]NEXTFILE** debug command.

]NEXTFILE takes zero or more *capability arguments*. Each argument must begin with either **HAVE-** or **NO-**. If the condition expressed by any argument is *not* met, the command returns immediately without effect. If *all* conditions are met, it calls **IO_Files::next_file()**, which closes the current testcase file and advances to the next one in the batch.

The recognised capabilities are:

```
□FFT  GTK  GUI  □PNG  POSTGRES  □PLOT  □RE  □SQL  SQLITE3  X11  XCB
```

Examples:

```
]NEXTFILE NO-□SQL      ⍝ skip if □SQL was not compiled in
]NEXTFILE HAVE-GTK     ⍝ skip if GTK is present (ASCII-only section)
]NEXTFILE NO-GUI       ⍝ skip if neither GTK nor XCB is available
```

The arguments are case-insensitive after the **HAVE-** or **NO-** prefix but the prefix itself *must* be in upper case. A bare word such as **no-GUI** (lower-case **no**) is not recognised; the command prints an "unknown capability" warning to CERR and continues without skipping — a silent no-op that is easy to introduce by mistake.

Typical usage places **]NEXTFILE** at a natural dividing point in the file, with build-independent tests before it and build-specific tests after. When the build does not have the required capability, the interpreter skips to the next file after the common tests; when it does, execution continues into the capability-specific tests.

3.6. Declaring Expected Errors:]EXPECT

]EXPECT n is easy to misread as "the previous *n* APL errors (**RANK ERROR**, **DOMAIN ERROR**, …) are expected." It is not. An ordinary APL error such as **DOMAIN ERROR** from **÷0** is **normal** testcase content: the interpreter prints the error message and a caret line exactly like any other output, and that text is checked the same way as any other expected output — by the ordinary line-by-line diff. A **.tc** file that only ever produces routine, correctly-captured APL errors never needs **]EXPECT** at all; most testcases with **DOMAIN ERROR** in their expected output have no **]EXPECT** anywhere in the file.

What **]EXPECT n** actually checks is the number of *testcase-mechanism* errors — errors internal to the test harness itself, counted separately from the ordinary expected-output diff — accumulated so far in the file, since its start or since the last checkpoint that matched. **]EXPECT** is dispatched through the same immediate-command table as **]NEXTFILE** or **]BOXING** (**src/Command.def**), so it runs the instant the interpreter reaches that line. **IO_Files::expect_apl_errors()** (**src/IO_Files.cc**) compares *n* against the internal **apl_errors** counter **as it stands at that moment**: if the counter is at most *n*, it is reset to zero; if it exceeds *n*, it is left untouched and whatever value it holds carries forward, eventually folding into the file's total error count in **summary.log** — a discrepancy is still a test failure, just not the kind a reader would guess from the name. For **]EXPECT n** to actually absorb a batch of errors, it must therefore be placed **after** the code that raises them, not before.

Some errors occur only on some platforms, but not on others. **]EXPECT** explicitly tolerates a (typically small) number of errors before the testcase file status becomes *failed*.

The errors this counter actually tracks are rare and specific — not **DOMAIN ERROR**/**RANK ERROR** text, which is normal, diff-checked content:

- an APL error that reaches the top level without ever having its error text printed (**Error::print_em()**/**print()** never called, so **Error::get_print_loc()** is still null when **Workspace::execute()** catches it — see **src/Workspace.cc**);
- **)SI** left non-empty at the end of a testcase file;
- a stale or incomplete **Value** or index expression detected by the automatic end-of-file check (**IO_Files::end_of_current_file()**).

The last two can only be detected once the file is fully read, which is also the only point at which **]EXPECT** could still be pending — so a **]EXPECT** meant to absorb those specifically only works if it is the *last* thing in the file. In practice, **apl_errors** is rarely nonzero at all under current, already-fixed behaviour: **src/testcases/Quad_ES.tc** still carries an **]EXPECT 1** next to a **⎕ES 0 1** call that once triggered an internal crash (Blake McBride, Bugs28 #71) — but running the file with and without that **]EXPECT** line produces the identical result (0 errors either way), confirming the declaration is now inert, kept as a documentation marker for a bug that no longer reproduces. **COPY.tc** records the same pattern explicitly, noting in a comment that a fix elsewhere means two former call sites "neither need[] **]EXPECT** any more."

3.7. Regenerating Expected Output

When a bug fix or deliberate behaviour change makes the existing expected output wrong, the expected output needs to be updated. The workflow has four steps.

Step 1. Run the test to produce a fresh **.tc.log** file:

```
src/apl --id 1010 -T src/testcases/Foo.tc
```

Step 2. Inspect the log. Confirm that the actual output shown in the **apl:** lines is correct — that is, that the new behaviour is intentional and not a new bug.

Step 3. Run **accept_tc.py** to overwrite the **.tc** file with a version whose expected output matches the log:

```
cd src/testcases
./accept_tc.py Foo.tc
```

The script reads **Foo.tc.log**, extracts every **---->** input line and every **== ` matching-output line and every `apl:** mismatch line, and writes a new **Foo.tc** that will pass the test.

Step 4. Check whether any line's output is legitimately variable for the **first** time — that is, has no wildcard in the previous expected output for **accept_tc.py** to preserve — and add a wildcard for it by hand. This is the one case the script cannot handle on its own: it preserves an **existing** wildcard whenever the new output still matches it, but nothing short of the developer's own judgement can tell it that a field that was always stable is now variable and needs a wildcard for the first time.

A convenience wrapper, **`accept_failures.py`**, automates step 3 for all failing tests at once:

```
cd src/testcases
./accept_failures.py          # regenerate all failing testcases
./accept_failures.py --all    # regenerate all testcases, even passing ones
```

The **`--all`** form is useful after a large systematic change that shifts output across many files. In both cases, checking for newly-variable output that needs its first wildcard (Step 4) is still a subsequent manual step.

The **`testcases_3/`** directory contains symbolic links to the files in **`testcases/`**, not copies. Editing a file in **`testcases/`** automatically updates the corresponding **`testcases_3/`** entry; there is never a need to edit both. **`accept_tc.py`** should always be run from **`testcases/`** so that it edits the real file and not a symlink.

3.8. ⎕RVAL and Property-Based Fuzzing

Hand-written **`.tc`** files test what somebody thought to test. For roughly seventy scalar and non-scalar primitives, across every combination of rank, shape, and cell type the standard allows, that ceiling is reached quickly. Nobody sits down and enumerates "**`take an axis of a rank-4 nested array past its own length`**" as a testcase to write by hand. **`⎕RVAL`** — a ⎕-function specific to GNU APL, compiled into every build (**`Quad_RVAL.cc`** is in **`common_SOURCES`**, not gated behind any configure option) — exists to generate exactly those cases automatically. It checks them against a machine-readable statement of what the ISO standard requires, and keeps doing so, unattended, every night.

3.8.1. The ⎕RVAL Subfunctions

⎕RVAL is dispatched like every other quad function, through **X ⎕RVAL B** where **X** selects one of eight subfunctions:

Table 6. The eight ⎕RVAL subfunctions

X	Name	Effect
0	**state**	get (**B** empty) or set (**B** non-empty) the generator's seed
1	**rank**	set the rank distribution for subsequently generated values
2	**shape**	set the shape distribution
3	**type**	set the data-type distribution (char/int/ float/complex/nested)
4	**depth**	set the maximum nesting depth
5	**ecount**	set the maximum element count (**⍴∈Z**) per generated value
6	**primitives**	return the primitive arity/stimulus/ constraint table (§below)
7	**conform**	generate a value that conforms to **B** (scalar or exactly **⍴B**)

Subfunctions 0–5 were the original design: set a rank/shape/type/ depth/ ecount distribution once, then call plain **⎕RVAL B** (**B** a 0–5-element vector overriding the distribution for one call) to draw a random value from it. **wslib3/INVARIANTS.apl**, the workspace this book's earlier editions described as GNU APL's fuzzing story, still uses exactly this: generate a random array, check that **⍴⍴A** is a one-element vector, that **(⍴A)[1]⌷A** selects the right element, and so on. These are universal algebraic laws that hold for *any* array, so no per-primitive bookkeeping is needed.

Subfunctions 6 and 7 are newer, and exist for a harder problem: laws that depend on which primitive you are looking at. **(⍴Z)≡⍴B** is the correct postcondition for **⌊B**, wrong for **⍪B** (Table always returns rank 2), and only sometimes right for **A+B** (only when **A** is scalar or already shaped like **B**). Testing per-primitive laws needs a per-primitive table, and generating arguments that actually reach the interesting code path — rather than failing immediately on a shape mismatch — needs generation that knows what each primitive's precondition is. That table is **src/rval.def**; the code that walks it is **src/testcases/AllPrimitives.tc**.

3.8.2. `rval.def`: One Row Per Primitive

`rval.def` is a plain data file, pulled into **`Quad_RVAL.cc`** via **`#include`** through a macro, one line per primitive:

```
prim_def("Z←A+B", ¯3 ¯100 (0 1 1 1 0) 3 RE, ~B, , "(⍴Z)≡⍴B")
prim_def("Z←⍪B",  ¯3 ¯100 (1 1 1 1 1) 3 RE, ,  , "(⍴⍴Z)≡,2")
```

Five fields, in the order the header comment documents them:

- **`arity`** — the APL expression to evaluate, e.g. **`"Z←A+B"`**;
- **`B_stim`** — a **`⎕RVAL`** generation spec for the right argument;
- **`A_stim`** — a spec, or blank (meaning **`0`**, i.e. monadic), for the left argument — this is the **precondition**: what **`A`** must look like for the primitive to have a defined result at all;
- **`X_stim`** — likewise for the axis;
- **`Z_constraint`** — an APL boolean expression in terms of **`A`**, **`B`**, **`X`**, and **`Z`**, the **postcondition** that must hold once the primitive has run.

The macro that expands **`prim_def`** swaps the second and third fields into the runtime table (**`{ arity, #a_stim, #b_stim, #x_stim, constraint }`**). So the column order a reader sees via **`6 ⎕RVAL 0`** (arity, A_stim, B_stim, X_stim, constraint) is **not** the order the call site lists them in — a trap for anyone reading **`rval.def`** next to code that consumes its output. **`RE`** is a literal placeholder token, not an APL name; the driver substitutes a real element-count budget for it before ever **`⍎`**-ing a stim string. **`RE`** left unresolved is simply a **`VALUE ERROR`** waiting to happen (an actual bug found and fixed during development — see below).

3.8.3. `CONFORM_A`: Generating Arguments That Reach the Postcondition

`B_stim`/**`A_stim`**/**`X_stim`** are unquoted C preprocessor tokens (the macro stringifies them with **`#`**), so they cannot contain arbitrary APL — glyphs like **`⊃`**, **`⍴`**, or **`↓`** are not valid preprocessor tokens outside a quoted string. For a precondition as simple as "**`A must be a non-negative int`**" that is no obstacle; a bare **`⎕RVAL`** spec vector says that directly. For a precondition like "**`A must be a rank-1 vector whose length equals B's last axis`**" (dyadic **`A/B`**, compress), no **`⎕RVAL`** spec vector can express the dependency on **`B`**'s actual, just-generated shape.

`A_stim` fields for such rows are instead a bare ASCII marker string — **`"~BC"`**, **`"~BIJ"`**, and so on — recognized by a real APL function, **`MARKER CONFORM_A B`**, that generates **`A`** (or **`X`**) as a proper function of **`B`**'s actual shape. As of this writing there are twenty markers, covering every family of primitive whose precondition genuinely depends on the other argument's shape. Examples include same-rank-minus-an-axis (**`~BL`**/**`~BF`**, for **`,B`**/**`⍪B`**- adjacent laminate/catenate forms), a count vector

matching a specific axis (**~BC**/**~BCF**, for **A/B**/**A≠B**), and a valid index into a specific axis (**~BI**, for **A⌷B**). The driving design principle: generate **B** first, then derive **A** from **B**'s real, already-known shape — never generate **A** and **B** independently and hope they happen to fit.

Two markers exist purely for interpreter-safety reasons rather than APL semantics. **AρB** and **A↑B** both need **A** to be an independent, short (0–8 element) integer vector — nothing about **B**'s shape constrains it. Left unconstrained, **⎕RVAL**'s ordinary integer generator draws each element from the **full `int64`** range, and multiplying eight such values together as a **`Shape`**'s ravel volume overflows **`int64`** — silently, since **`Shape::add_shape_item()`**'s volume multiplication has no overflow check. That is a deliberate choice: it sits on a hot path, and the interpreter's own contract is that callers never hand it values whose product can overflow. The overflow crashed **A↑B** outright and silently corrupted **AρB**'s result (see below). Marker **~SC** sidesteps the whole problem elegantly: instead of drawing eight independent large integers, it asks **⎕RVAL** to generate a **throwaway array** — any array — and uses *that array's own shape* as **A**. **⎕RVAL** already clips a generated array's shape internally so that **×/shape ≤** the configured element-count budget. Since **×/R ≥ ⌈/R** for any non-negative **R**, that same bound on the product also bounds every individual element — no separate per-element cap needed, and the safety margin is expressed in the same units (**ecount**) as everything else **⎕RVAL** already does.

3.8.4. The Driver: `ON_ERROR`, `ALL_PRIMITIVES`, `ONE_PRIMITIVE`

Three functions in **`AllPrimitives.tc`** tie the table to actual test runs. **`ONE_PRIMITIVE`** picks one random row, generates **B** (then **A**, conforming to it if the row asks for that, then **X**), executes the arity expression, and evaluates the postcondition — returning a structured result (**ZZ.OK**, plus the exact **A**/**B**/**X**/**OPER** used). **`ALL_PRIMITIVES`** calls it in a loop for a configured number of seconds, counting violations. **`ON_ERROR`** turns a violation into a self-contained reproducer. Using **10 ⎕CR** to serialize **A**, **B**, and **X** back into literal APL assignment statements and appending the failing expression, it writes a **`.apl`** script. Run months later on a different machine, with none of this context loaded, that script reconstructs the exact failing case and re-executes it.

3.8.5. Bugs Found This Way

Both the mechanism and its output are new enough that a short, honest account of what it has actually caught is more convincing than an abstract description. In roughly a day of use, this harness found and led to fixes for:

- A stale-label crash: a defined function whose last line is a bare, empty label loses track of that label's address whenever an earlier **{...}** dfn call shrinks the compiled function body — a subsequent jump reads past the end of the compiled code and corrupts the parser's stack.

- A missing **`check_value()`** call: **`⊃[X]B`** crashed with an internal "incomplete value" assertion whenever **`B`** had rank 3 or higher and an empty axis, because dyadic transpose's rank-≥3 code path has an early return for empty results that never marked the value complete.
- Two independent bugs in **`Bif_F12_TAKE`**'s axis-take fill logic (**`A↑[X]B`**): the offset used to fetch the overtake fill/prototype cell summed the wrong set of axes (contradicting the function's own doc comment), and separately mixed up natural axis numbering with the transposed numbering **`Shape`** uses internally. Together they caused an out-of-bounds read into **`B`**'s own ravel once the overtake position walked far enough.
- An assertion failure in **`A!B`** (binomial): the negative-binomial identities used for negative **`N`**/**`K`** do an internal subtraction that overflows **`int64`** for extreme arguments, violating **`K33_binomial`**'s own **`Assert(N>=K)`**.
- A crash in **`A f/B`** (n-wise reduce): the left argument was narrowed from a 64-bit **`APL_Integer`** to a plain 32-bit **`int`** before a bounds check. This occasionally landed on a **negative** truncated value, which trivially passed the "not too large" check, while the untruncated, huge, unchecked original value went on to compute a wildly out-of-bounds ravel offset.

None of these are contrived; each was the first thing the fuzzer happened to generate that exercised the gap. Several were assertion failures or SEGVs — the harness does not merely catch wrong *answers*, it catches the interpreter failing to produce an answer at all.

3.8.6. Running It Nightly

`ALL_PRIMITIVES` now runs unattended every night on a build server. Any newly discovered failure — reproducer script attached — lands in the maintainer's inbox each morning, the same way the multi-configuration build matrix (Chapter 2) does for build failures. Coverage is uneven and improving: of the 112 rows in **`rval.def`**, 50 use a **`CONFORM_A`** marker (or an equivalent direct **`~B`** spec) as of this writing. The remaining 62 still generate their arguments independently and mostly fail on a shape mismatch before their postcondition is ever evaluated. Extending conformance rules to those rows, one primitive family at a time, is ongoing work. Unusually for this project, it is work an AI assistant has been doing largely independently: given the rough direction, it wrote and tested the **`6 ⌷RVAL`** C++ code, the **`AllPrimitives.tc`** driver, and (not originally planned, but found necessary while testing subfunction 6) subfunction 7 itself.

3.9. Profiling and Coverage Testing

The testcase suite doubles as the stimulus for two standard measurement tools that the compiler provides: **`gprof`** for execution profiling and **`gcov`** for source-line coverage analysis. Both are activated by passing options to **`./configure`** itself — not environment variables set before it — and need no changes to testcase files.

3.9.1. gprof — Execution Profiler

`gprof` records how much CPU time each function consumes and builds a call graph. Enable it by passing **`GPROF_WANTED=yes`** to **`./configure`**:

```
./configure GPROF_WANTED=yes
make -C src
```

The flag causes the compiler to add **`-pg`** to every compilation and link unit. When the interpreter runs it writes a binary **`gmon.out`** file in the current directory on exit. Feed that file to **`gprof`** together with the unstripped executable to produce a human-readable profile:

```
cd src
./apl --id 1010 -T testcases/*.tc   # produces gmon.out
gprof apl gmon.out > profile.txt
less profile.txt
```

The profile report has two parts: a flat profile (function name, call count, self time, cumulative time) and a call graph (who called whom and how much time was charged to each edge). A typical use is to run **`make -C src test3`** as the stimulus, let all 260+ testcases execute, and then look at which internal functions dominate the flat profile.

Profiling instrumentation adds overhead; do not use a **`GPROF_WANTED=yes`** build for performance benchmarking. Rebuild without the flag for any timing comparison.

3.9.2. A Worked Example: Where test3's Cycles Go

Running **`test3`** itself under **`gprof`** (2026-07) is a useful exercise precisely because it stresses a broad, representative cross-section of the interpreter rather than one hand-picked benchmark expression. A single clean pass is too short to sample meaningfully: 261 files execute in a few hundred milliseconds of profiler-eligible CPU time, yielding barely thirty 10 ms samples. The useful technique instead is to repeat the suite several times within one process (**`-T`** accepts the same file list concatenated N times) to accumulate enough samples. That does mean accepting a small number of cross-repetition workspace-state errors, which come from running testcases designed for a single pass more than once — a version of the same testcase-interference this

chapter already discussed. A 15×-repeated run accumulates roughly 400 samples — still modest, but enough for the coarse question this section asks: not "which single function is slowest" but "which **layer** of the interpreter dominates."

Classifying every function in the flat profile by the source file that defines it, and rolling the self-time up into four layers, gives the following breakdown. The four layers are **`Parser.cc`**/**`Tokenizer.cc`**/**`Avec.cc`** (parsing), **`Prefix.cc`** (interpretation — deciding what to execute next), **`Bif_*.cc`**/**`Quad_*.cc`** (execution — the primitives themselves), and everything else (the **`Cell`**/**`Ravel`**/**`Value`**/**`SymbolTable`** data-model layer Chapters 4–6 and 15 cover):

Table 7. Self-time distribution across `test3`, 15 runs, ~400 samples

Layer	% Time	Representative functions
Data-model support (**`Cell`**/**`Ravel`**/**`Value`**/**`SymbolTable`**/…)	~81 %	**`cValue::get_cravel()`**, **`Ravel::cell_fetcher()`**, **`Cell::is_pointer_cell()`**
Execution (**`Bif_*`**/**`Quad_*`**)	~7 %	**`Bif_F12_STILE::eval_B()`**, **`Bif_OPER2_OUTER::PF_scalar_outer_product()`**
Interpretation (**`Prefix.cc`**)	~4 %	**`Prefix::reduce_body()`**, **`Prefix::reduce_A_F_B_()`**
Parsing (**`Parser.cc`**/**`Tokenizer.cc`**)	~4 %	**`Parser::parse_statement()`**, **`Tokenizer::tokenize()`**
Test-harness I/O (reading **`.tc`** files, formatting diagnostics)	~3 %	**`IO_Files::get_file_line()`**, **`PrintBuffer::do_PrintBuffer()`**

Two things stand out. First, the layer most people mean by "the interpreter" in casual speech — the code that reads a statement and decides what to do with it, **`Parser.cc`** plus **`Prefix.cc`** — accounts for only about 8 % of self-time. Second, and more tellingly, over 80 % of the time lands not inside a primitive's own **`eval_*`** body but in the small accessor and helper functions of the array/cell layer those bodies call into. **`Ravel::cell_fetcher()`**, **`Ravel::int64_fetcher()`**, and siblings are, by name, exactly the packed-ravel materialisation functions §4.2.3 (Chapter 4) documents. This profile is, among other things, live confirmation that a realistic workload spends its cycles where Chapter 4 predicted it would.

Looking inside **`Prefix.cc`** specifically sharpens the picture further. By raw call count, the shift-side bookkeeping — **`Prefix::do_shift()`**, **`push_Symbol()`**, **`set_action()`** — is invoked about twice as often as every **`reduce_XXX()`** grammar-rule handler combined (calls happen on every token shifted, but only when a phrase is recognized). By self-time the ranking inverts: **`reduce_XXX()`** accounts for roughly seven-

eighths of **Prefix.cc**'s own cost despite the smaller call count. That is on the order of 45 ns per call versus 3 ns for the shift-side bookkeeping — about 14× more expensive per call. That asymmetry is exactly what the split predicts: shifting a token is cheap, uniform bookkeeping, while a **reduce_XXX()** is where a completed grammar phrase actually gets **evaluated**. **reduce_body()**, the handler that fires once a whole statement is done (committing its result, driving ⎕← output, SI bookkeeping), alone accounts for about two-thirds of that group's cost. That puts it well ahead of narrower reducers like **reduce_A_F_B_()**, which mostly just forward to a primitive's **eval_AB()** and return.

This worked example returns in §3.13 at the end of this chapter, where the same numbers are read for a different question: not which layer is slowest, but how much of GNU APL's total runtime is "interpretation tax" at all.

3.9.3. gcov — Source-Line Coverage

gcov records whether each source line was executed at least once. Enable it with **GCOV_WANTED=yes**:

```
./configure GCOV_WANTED=yes
make -C src
```

The flag passes **--coverage** to the compiler, which compiles in instrumentation and links the **gcov** runtime library. After a run, **.gcda** data files accumulate in the same directories as the **.o** object files. Use **gcov** to generate a per-source-file report:

```
cd src
./apl --id 1010 -T testcases/*.tc   # fills .gcda files
gcov Value.cc                          # produces Value.cc.gcov
```

The **.gcov** file is an annotated copy of the source: each line is prefixed with the execution count (**#####** for never-executed lines). The testcase suite is the natural stimulus because it exercises the interpreter through its full APL-level surface; running it under **gcov** reveals which internal paths the existing tests do and do not reach.

Running the full test3 suite under **gcov** produces roughly 70–75 % line coverage for the core interpreter files (e.g., **Prefix.cc** ≈ 71 %, **ComplexCell.cc** ≈ 69 %). Lines that remain uncovered after the full suite are candidates for new targeted testcases; **src/testcases/Coverage.tc** was written specifically to exercise low-coverage paths identified this way.

For accurate results, add **-O0** to suppress inlining optimisations that can make individual lines unreachable or merge several source lines into one machine instruction:

```
./configure GCOV_WANTED=yes CXXFLAGS="-O0 -g"
make -C src
```

GPROF_WANTED and **GCOV_WANTED** can technically be set at the same time (each simply appends its flag to **CXXFLAGS**), but their combined overhead makes the result less useful than either alone; set at most one per build.

3.10. Performance Probes

GNU APL has a second, lower-level measurement facility: software probes. A probe is a pair of calls bracketing a region of C++ code; the CPU cycle counter is read at each call and the difference is stored. After the region has been executed up to 20 times the stored cycle deltas can be retrieved from APL with **⎕FIO**.

Probes are always compiled in — they require no configure flag — but they are lightweight enough that the unused probes add no measurable overhead.

3.10.1. Why 20 Measurements?

The choice of 20 (**PROBE_LEN = 20** in **src/Common.hh**) is not a round number pulled from thin air: it is grounded in a theorem from probability theory, and the author's background in mathematics before computer science shows here.

The probe mechanism itself is a GNU APL invention, not a carry-over from an earlier project. But the reasoning behind the choice of 20 reaches back to the author's work on the DATIS project (circa 1990): at that time CPUs were fully deterministic. Given the exact sequence of opcodes and their operand types, the number of clock cycles consumed was fixed and could be derived from the instruction-set manual. A single measurement was therefore exact, and a handful of runs mainly served as a sanity check.

The statistical motivation rests on two theorems.

The **Central Limit Theorem** (CLT) states that the sample mean of n independent, identically distributed random variables with finite mean and variance converges in distribution to a normal distribution as $n \to \infty$, regardless of the shape of the original distribution. It says the normal approximation *eventually* holds, but not how fast.

The **Berry-Esseen theorem** (independently proved by Andrew Berry in 1941 and Carl-Gustav Esseen in 1942) fills that gap with a quantitative bound. For a distribution with standard deviation σ and third absolute central moment ρ, the maximum deviation of the sample-mean CDF from the normal CDF is bounded by $C \cdot \rho / (\sigma^3 \cdot \sqrt{n})$, where $C \approx 0.4748$. For unimodal, not-strongly-skewed distributions — which timing data typically is once outliers are removed — this error drops below a few percent by $n \approx 20$–30. That is where the statistician's traditional rule of thumb "n ≥ 20 is large enough" comes from, and that is where **PROBE_LEN = 20** comes from.

Modern CPUs break the determinism assumption in three ways, on two different timescales. Caches (instruction, data, TLB) warm up fast: the *first* execution of a code path pays the cache-cold penalty, but by the second execution the relevant lines are resident and stay that way unless something else evicts them. Branch-predictor history is slower to settle. A branch predictor does not learn a branch's outcome from a single observation. It adjusts a saturating counter (or, on modern cores, a whole table of correlated history) toward "taken" or "not taken" one increment at a time. So a branch that has only been seen once or twice may still mispredict on the next few executions even though the cache is already warm. This book's author measured the gap between the two directly, in the early days of GNU APL's parallel-execution work. Cache effects settled after one warm-up run as expected, but the timing of the same probed region kept drifting for another three or four runs before it converged. Branch-predictor state, not the cache, was still adapting. Out-of-order execution and cache-line contention then add ordinary run-to-run variation on top of both, even once caches and predictors have both settled.

The standard practice that addresses all of this is:

1. Run the probed region repeatedly before measuring — one iteration is enough for the caches, but branch-predictor state has been observed to need three or four before it stops drifting; these *warm-up* iterations are discarded.
2. Collect a batch of measurements (20 is enough for the CLT to apply).
3. Average the batch; optionally also compute the standard deviation to check for outliers.

GNU APL ships with APL workspaces that implement exactly this protocol. **`src/workspaces/Performance.apl`** and the **`src/testcases/Performance.pt`** timing harness both run each expression enough times to amortise the cold-start cost and then report averages. The raw cycle deltas returned by **`p ⎕FIO ¯3`** can be processed in APL to apply the same treatment to custom probes.

3.10.2. The Probe API

The **`Probe`** class is defined in **`src/Common.hh`**. There are 100 probe slots, indexed 0–99. Five named aliases are provided for the slots nearest the top of the array: **`Probe::P_1`** through **`Probe::P_5`** (slots 99, 98, 97, 96, 95). The remaining slots 0–94 form a vector reachable as **`Probe::P0`** through **`Probe::probes[94]`**.

Each slot stores up to 20 cycle-count pairs (**`PROBE_LEN = 20`**). When more than 20 measurements are taken, the excess is silently dropped.

The **Probe** methods used in C++ are:

Table 8. The Probe class's C++ methods

Method	Effect
Probe::P_1.start()	Read the cycle counter and store it as the *from* value of the next slot in **P_1**.
Probe::P_1.stop()	Read the cycle counter and store it as the *to* value; advance the slot index.
Probe::init(p)	Clear all measurements in slot **p**.
Probe::init_all()	Clear all 100 slots.

3.10.3. Inserting a Probe in the Source

Place **start()** and **stop()** around the code region of interest:

```
// before the region of interest:
Probe::P_1.start();

// ... the code to measure ...

Probe::P_1.stop();
```

After each **stop()** the difference **cycles_to − cycles_from** is available as the next measurement in slot **P_1**. Up to 20 consecutive start/stop pairs are recorded before the slot wraps.

3.10.4. Reading Results from APL

⎕FIO provides three probe-related operations:

```
⎕FIO ¯1        ⍝ return the raw CPU cycle counter (one integer)
⎕FIO ¯3        ⍝ return 1 if rdtsc is available, 2 if using gettimeofday
⎕FIO ¯4        ⍝ clear all 100 probe slots; returns 0
p ⎕FIO ¯3      ⍝ return cycle deltas for probe slot p, then clear slot p
```

The last form — dyadic **⎕FIO** with right argument **¯3** — is the primary read-back operation. **p** is the probe slot index (0–99; negative indices count from the top, so **¯1** = slot 99 = **P_1**). It returns a vector of up to 20 integers, each being the **stop − start** cycle count for one measurement, and then resets the slot.

A typical APL session that exercises probe **P_1** looks like this:

```
⎕FIO ¯4            ⍝ clear all probes before starting
⍝ ... run the APL expression that exercises the instrumented code ...
deltas ← ¯1 ⎕FIO ¯3   ⍝ read probe P_1 (slot 99 = index ¯1) and clear it
⌊deltas÷(⎕FIO ¯2)÷1e6  ⍝ convert cycles to microseconds using CPU frequency
```

`⎕FIO ¯2` returns the CPU frequency in Hz; dividing cycles by `(⎕FIO ¯2)÷1e6` converts to microseconds.

3.10.5. The]PSTAT Command

For the built-in performance counters that GNU APL already instruments (scalar cell functions, output, inner and outer product, etc.) there is a simpler route: the `]PSTAT` debug command. It requires a build with `PERFORMANCE_COUNTERS_WANTED=yes`:

```
./configure PERFORMANCE_COUNTERS_WANTED=yes
make -C src
```

This flag activates the `PERFORMANCE_START` / `PERFORMANCE_END` macro pairs that already surround critical sections throughout the source (for example in `ScalarFunction.cc` and `DiffOut.cc`). Without the flag those macros expand to nothing.

After running the interpreter, `]PSTAT` prints a table of all instrumented operations with their call counts and average cycle costs:

```
      ]PSTAT          ⍝ print all counters
      ]PSTAT CLEAR    ⍝ reset all counters
      ]PSTAT SAVE     ⍝ write counters to ./PerformanceData.def
```

`]PSTAT SAVE` writes a `.def`-format file that can be loaded by the APL workspace `src/workspaces/Performance.apl` to produce gnuplot charts (see `src/workspaces/Performance.gnuplot`).

3.10.6. The `make perf` Target

The `perf` target in `src/Makefile.am` runs the dedicated performance testcase file:

```
make -C src perf
```

This invokes:

```
./apl --id 1010 --TM 3 -T testcases/Performance.pt
```

The `.pt` file (Performance Test) is an APL program — not a `.tc` testcase — that defines timing harness functions (`MAKE_FUT`, `RUN_PERF`, `DELTA`, `GO`) and benchmarks a set of common APL expressions. It runs each expression enough times to get a stable wall-clock reading, then prints a summary table of time-per-operation alongside iteration counts. Unlike `]PSTAT`, this measures end-to-end APL-level latency rather than internal C++ cycle counts; the two approaches are complementary.

3.10.7. A Worked Example: HASH vs. TREE Phrase Lookup

Prefix.cc's shift-reduce parser (Chapter 9) calls one function, **find_best_phrase()**, once per shift and once per reduce, to decide whether the tokens currently on the stack match one of GNU APL's ~130 grammar phrases. It is declared **inline**, so under normal compilation it does not exist as a separate symbol at all. The compiler fuses its body directly into **reduce_body()**'s loop, which is exactly why it never appears as its own line in **gprof**'s output (Table 3-3, earlier in this chapter, attributes its cost to **reduce_body()** along with everything else that loop does).

Until recently GNU APL carried **two** implementations of this lookup, selected at code-generation time by **tools/phrase_gen** (plain invocation for a minimal perfect hash table, **phrase_gen -t** for a search tree). Only the hash-table form was actually shipped. **phrase_gen** has defaulted to hash mode for long enough that the tree mode had quietly stopped compiling: it called **Prefix::size()**, a method that no longer existed (only **ssize()** does). That in itself is a small, useful lesson about conditionally-compiled code that nothing in the test suite or CI ever builds. It rots exactly like any other code that never runs, just silently, because nobody notices until someone goes looking.

The question worth asking before deleting the tree variant outright was whether it was actually slower, or merely unmaintained — this book's author's own expectation was that the hash table would win. Answering it meant applying this chapter's own probe methodology to a real, live design question. **Probe::P_1** was bracketed around the **find_best_phrase()** call site, and each build was measured with the full protocol above: four discarded warm-up runs (not one; see the branch-predictor discussion earlier in this section), then a batch of ~20 real measurements, repeated across several batches:

Table 9. find_best_phrase(), cycles per call, five batches per build

Build	Batch means	Grand mean
HASH (shipped)	230, 120, 235, 217, 332	227 (≈71 ns)
TREE (regenerated for this test)	211, 251, 252, 209, 257	236 (≈74 ns)

The two grand means differ by about 4% — and each build's own batch-to-batch spread (HASH ranges from 120 to 332, a 2.7× swing) is far wider than that 4%. The honest reading is that the two approaches are statistically indistinguishable on this measurement. Whatever separates a handful of rolling-hash computations plus up to four table probes from a trie walk at most four levels deep, both are bounded by the same small constant (the longest phrase is 4 tokens). At that scale, implementation-detail differences are smaller than ordinary system

noise on a shared, unpinned machine. Structural reasoning would have predicted this outcome before any measurement was taken; what the probes added was the confidence to act on it rather than merely assume it.

With no measurable difference and one implementation already bit-rotted, the tree variant was removed outright. **`find_best_phrase()`**, the code generator's tree-emitting branch, and the **`Phrase`** struct's **`sub_nodes[]`** array (19 bytes × the hash table's 519 entries, ≈9.6 KB that HASH mode allocated but never read) all went in the same change. **`PHRASE_MODU`** — the size of that hash table, chosen so all ~130 real phrases land in distinct slots with no collisions — was untouched; removing TREE was a subtraction, not a redesign.

That outcome is worth connecting back to §3.10.1 above. The Central Limit Theorem and Berry-Esseen bound there were not mathematical decoration attached to a simple counting tool. They are what made this specific "no difference" reading trustworthy enough to delete working code on the strength of it, rather than leaving two implementations around indefinitely out of uncertainty. The very first properly warmed-up batch of each, taken in isolation, already illustrates why: HASH's first batch averaged 230 cycles against TREE's first batch at 211. TREE was apparently **faster** — the opposite of this book's author's own expectation, and a plausible-looking result on its own. Only after collecting several more independent batches per build did the honest picture emerge. HASH's own batch-to-batch spread alone (120 to 332) dwarfs the gap between any two single readings, and the "TREE is faster" conclusion evaporates into noise the moment a second batch is taken. A single batch, however properly warmed up, was not enough; it took exactly the CLT-and-repetition discipline §3.10.1 above describes to tell a real effect from a lucky first draw. This is performance measurement's practical payoff in general, not just the probe API's. A real, if modest, engineering decision — delete this code, keep that one, stop guessing which is faster — was made correctly instead of by gut feeling. The methodology was sound enough to trust, even when the honest answer was "it doesn't matter."

3.11. Notes

The **`.tc`** format dates from the earliest versions of GNU APL and has been remarkably stable. Its simplicity — plain text, no special runner binary, executable directly by the interpreter — means that writing a new testcase requires no infrastructure knowledge at all. Compose APL expressions in a text editor, paste their expected output below each one, and the file is done.

The wildcard mechanism was added when the test suite first encountered platform-dependent output. The choice of superscript digits as wildcards is idiosyncratic but practical: they are valid Unicode, they do not appear in any APL output except intentionally,

and they are visually distinct from both APL glyphs and ordinary ASCII. A reader who sees `⍬` in an expected-output line immediately knows it is a wildcard, not an APL character that happened to be output.

Early versions of **`accept_tc.py`** stripped wildcards from every mismatched line it regenerated, even when the wildcard itself was not the cause of the mismatch. The script now checks, for each regenerated line, whether the **previous** expected line still matches the new actual output once its wildcards are taken into account (see "**`Wildcards in Expected Output`**" above); if it does, the old wildcarded line is kept verbatim instead of being overwritten. This is safe specifically because a wildcard, by construction, never causes a genuine diff failure on its own — so "does the old wildcarded line still match" is a reliable test for "this is the same variable field, not a regression." Only a field that becomes variable for the **first** time — one with no prior wildcard for the script to find — still needs a wildcard added by hand after regeneration.

3.12. Excursus: Testing in the Large

Testing a "**`Hello, World!`**" program is trivial. There is one input — nothing, or a fixed command-line invocation — and one expected output, a known string. Any discrepancy is a bug. The entire test fits in a shell one-liner.

Scale this up to a real project: a compiler, an interpreter, a database engine, or an operating system. The number of observable behaviours grows combinatorially, not linearly. An APL interpreter evaluates arbitrary expressions built from roughly 70 primitive functions, six operator forms, nested arrays of arbitrary depth and shape, and a workspace mechanism that carries state across statements. The number of possible input programs is infinite; the number of interesting *corner cases* is at least several thousand. No engineer can hold this in their head and no informal check of a few examples constitutes adequate testing. This gap — between what can be verified by inspection and what must be verified by systematic process — is the subject of this section.

3.12.1. What Must Be Tested?

A convenient taxonomy for the things a test suite must cover:

Functional correctness is the obvious category: does **`2+2`** return **`4`**? Does **`⍴1 2 3`** return **`3`**? For an APL interpreter the functional specification is the ISO standard plus the documented GNU APL extensions, and every section of both documents is a source of testable claims.

Edge cases and boundary conditions are where most bugs live. Empty arrays (rank 0, shape **`0`**, or shape **`0 N`**), scalars treated as rank-0 arrays, the boundary between integer and floating-point representation, the behaviour of **`⌈/0`** (reduction over an empty vector), overflow, underflow — these are the inputs that are easy to overlook in an implementation but that real programs eventually hit.

Error paths must be tested explicitly. If an expression should signal **`DOMAIN ERROR`**, the test suite must confirm that it does, and that the error message is correct. Testing only the happy path is a well-known trap: an implementation that raises a **`RANK ERROR`** where a **`DOMAIN ERROR`** is required has a bug, even though neither is a "**`correct`**" result. GNU APL's **`]EXPECT`** mechanism exists precisely to make error-path testing first-class.

Regression prevention is perhaps the most important function of any mature test suite. Every time a bug is fixed, a testcase that would have caught the bug *before* the fix was written should be added to the suite. Over the lifetime of a project, the regression suite becomes a living record of every problem that was ever found and corrected. Without it, the same bug will be reintroduced — often by a refactoring that looks unrelated — and discovered again, at greater cost.

Configuration variants matter when the project compiles differently depending on available libraries or build options. For GNU APL these include **`⌻FFT`** (requires FFTW), **`⌻SQL`** (requires SQLite), rational numbers (**`RATIONAL_NUMBERS_WANTED`**), and platform endianness. A test that passes in the standard build but was never run in the **`develop`** or **`libapl`** configurations can mask a real defect. The nightly build matrix described in Chapter 2 is the answer to this: the test suite is run against every supported configuration on every revision.

The build and install path is a category that is easy to neglect because it lies outside the interpreter proper — and yet it is the first thing a new user exercises. A project whose source builds correctly from a clean checkout, installs into the expected locations, and runs after installation has passed a non-trivial end-to-end test. A project that produces a correct binary but installs it to the wrong prefix, forgets to install a required workspace file, or leaves a stale **`config.status`** that causes **`./configure`** to fail on a second run has a defect, even though no APL expression ever produced a wrong answer. For GNU APL, the build path test is embodied in the nightly matrix: the **`tar`** configuration extracts the distributed tarball from scratch and builds it, exactly as an end user would. If **`autoreconf`** is broken, or a required file is missing from **`EXTRA_DIST`**, or the installed **`libapl.so`** is missing a symbol, the nightly run catches it before a user does.

Nightly builds across multiple configurations amplify the value of the testcase suite by running it against every supported combination of configure options. A single-configuration CI catches regressions in the default build. A multi-configuration nightly build additionally catches regressions that only manifest when a feature is compiled out (a **`#ifdef`** branch that compiles but is never executed in the standard build), when optional libraries are present (a new **`⌻SQL`** code path that was never tested without SQLite), or when the build order matters (a

library that must be installed before a later configuration can link against it, as **`libapl`** must precede the **`erlang`** configuration). The cost is proportional to the number of configurations and the build time per configuration; with a single-core server the full matrix takes several hours, which is why it runs overnight rather than on every commit.

Performance is a softer category because it depends on the hardware. Absolute cycle counts are not portable assertions. What can be tested is that performance does not *regress*: if an operation that previously ran in 10 ms now takes 1 s, that is a signal worth catching, even if the absolute numbers are not meaningful. GNU APL addresses this with the **`make perf`** target and the probe mechanism, though neither is currently wired into an automatic regression-detection system.

3.12.2. Strategies and Tools

The software-engineering literature describes several complementary strategies that are used in practice.

Unit testing tests individual functions or classes in isolation, with all dependencies replaced by stubs or mocks. A unit test for **`IntCell::bif_add`** would call it directly from a C++ test harness — not through the interpreter — and check the returned cell value. The dominant C++ frameworks are GoogleTest (used by Chromium, LLVM, and Android), Catch2 (header-only, popular in embedded and games), and the older CppUnit. Unit testing catches implementation errors early and produces precise failure reports, but requires the test to be written in the same language as the implementation.

The author has deliberately chosen *not* to adopt unit testing for GNU APL, based on direct experience with the approach on a mid-sized C++ project. Almost every change to a function's implementation also required updating the corresponding unit test — effectively doubling the maintenance work for each change. More tellingly, when the number of bugs *actually caught* by those unit tests was tallied, the count was exactly zero. (This excludes failures where the unit test simply had not been updated after a code change and the compiler had already flagged the incompatibility.) The bugs that mattered were all detected at the integration level or by the compiler itself. GNU APL's integration-style **`.tc`** testcases have proven to be a better investment of the same effort: they exercise real behaviour, require no separate test harness, and produce precise, reproducible failures when something breaks.

Integration testing verifies that components work together correctly. GNU APL's **`.tc`** testcase files are integration tests: they exercise the tokenizer, parser, evaluator, and output formatter end-to-end. The test says nothing about which internal component failed — only that the observable APL output is wrong — but that is exactly the right granularity for detecting regressions caused by interactions between components.

Fuzzing (or fuzz testing) generates random or mutated inputs and feeds them to the program, looking for crashes, assertion failures, or sanitizer violations. Modern fuzzers such as AFL++ and libFuzzer are *coverage-guided*: they track which code paths each input exercises and mutate inputs to reach new paths, building up a corpus of interesting test cases automatically. A fuzzer running against an interpreter is a powerful tool for finding inputs that trigger undefined behaviour or unexpected exceptions. Neither AFL++ nor libFuzzer is currently applied to GNU APL's tokenizer or parser directly; adding that support would require a small harness that feeds arbitrary byte strings to the tokenizer and catches any unexpected exits.

GNU APL takes a related approach using APL itself, at the level of individual primitives rather than raw byte strings. The **⎕RVAL** system function generates random arrays of arbitrary rank, shape, element type, and (via **CONFORM_A**) arguments that structurally conform to a chosen primitive's precondition. It then checks the result against a postcondition taken directly from the ISO standard. This is **rval.def** and **AllPrimitives.tc**, described in full earlier in this chapter (see "**⎕RVAL and Property-Based Fuzzing**"); it has already found and led to fixes for several real crashes and assertion failures. The older **wslib3/INVARIANTS.apl** workspace predates this and takes a complementary, coarser-grained approach: universal algebraic laws that hold for *any* array regardless of which primitive produced it (**⍴⍴A** is always a one-element vector, **(⍴A)[1]⌷A** selects the correct element). These run under **make test4**.

Property-based testing (the approach popularised by Haskell's QuickCheck, now available in C++ as RapidCheck) expresses invariants as universally quantified properties — "**for all arrays A, `⍴⍴A** is a one-element vector`" — and generates hundreds of random instances to search for a counterexample. This is particularly natural for APL because many of the language's laws (the rank-duality laws, the scalar-extension rules, the conformability conditions) are expressible as closed-form algebraic identities. GNU APL now has two implementations of the idea at different granularities — **INVARIANTS.apl**'s universal laws and **rval.def**'s per-primitive pre/postcondition table — see the Fuzzing paragraph above. The per-primitive table already runs nightly (see "**Running It Nightly**" earlier in this chapter); wiring **INVARIANTS.apl** into the same nightly run would extend continuous property-based testing to the universal laws as well.

Continuous integration (CI) means that the test suite is run automatically on every commit, not just before major releases. A CI pipeline typically:

1. checks out the revision;
2. builds in one or more configurations;
3. runs the test suite;
4. reports the result within minutes.

If the suite is fast (under five minutes), developers can afford to run it before pushing. If it is slow, a subset is run pre-push and the full suite is run overnight. GitHub Actions, GitLab CI, Jenkins, and the GNU project's own Savannah build farm are all popular choices. GNU APL uses a nightly cron job on **`server72`** (described in Chapter 2) that builds all configured variants and emails the results — functionally equivalent to CI, though not triggered by individual commits.

3.12.3. Code Coverage as a Guide, Not a Goal

Code coverage measures how much of the source was executed by the test suite. Reaching 100 % line coverage is neither achievable nor necessary: some defensive branches guard against conditions that cannot happen given correct usage, and instrumenting every one of them would produce fragile tests that test the tests rather than the program.

The right way to use coverage is as a *discovery tool*: run the suite under **`gcov`**, look for clusters of uncovered lines, ask whether those lines represent:

- **paths that can never be reached** — candidates for removal (dead code);
- **paths that are reachable but not yet exercised** — candidates for new testcases;
- **paths that are only reachable through error injection** — acceptable gaps that should be documented.

For GNU APL the full testcase suite achieves roughly 70–75 % line coverage in the core interpreter files. Dead code found this way in a recent coverage pass included **`ComplexCell::bif_equal`** and two impossible parser phrases (**`LPAR F C RPAR`** and **`A M`**); all three were removed. The remaining uncovered lines are either error-handling branches or platform-specific code paths.

3.12.4. How GNU APL Compares

GNU APL is a single-maintainer open-source project of roughly 175,000 lines of C++. Comparing it with other projects in the same space is instructive.

Dyalog APL (commercial, closed-source) is the dominant modern APL implementation. Its test suite is internal and not publicly described, but Dyalog publishes conformance notes against the ISO standard and maintains a regression suite that covers its documented behaviour. The company has the resources of a dedicated QA team; the test suite is not available for public inspection or contribution.

GNU GCC and Clang/LLVM are the most heavily tested open-source compilers. GCC's test suite runs millions of test cases organised into separate suites for the C, C++, Fortran, and Ada front-ends; the DejaGnu framework drives them. LLVM uses the LIT (LLVM Integrated Tester) framework and also runs a separate suite of clang-format and clang-tidy tests. Both projects use automated coverage dashboards

(coverage.libreoffice.org, llvm.org/docs/TestingGuide.html) that track coverage over time. Both also use AFL-based fuzzing continuously on the compiler front-ends. These projects have hundreds of contributors and paid test infrastructure. GNU APL has one maintainer and a rented VPS.

Python (CPython) uses the **`unittest`** module to run roughly 500,000 assertions in a self-hosted test suite (**`Lib/test/`**). The suite covers the standard library as well as the interpreter core. CPython also runs under AddressSanitizer and MemorySanitizer on every CI run to catch memory errors, and under Valgrind periodically. Its buildbot farm tests every commit on GNU/Linux, macOS, Windows, FreeBSD, and several less-common platforms.

Interpreted APL dialects at the other end of the spectrum — ngn/APL (a few hundred lines of JavaScript) and dzaima/APL (a few thousand lines of Java) — typically have much smaller test suites or rely on the APL contest community to surface bugs.

In this landscape, GNU APL occupies a solid middle ground. Its ~270 hand-written testcase files are comprehensive by the standards of a single-maintainer project, cover the full APL standard surface, and include testcases that were written specifically to exercise low-coverage paths. As of this writing, they are complemented by the **`rval.def`**/ **`AllPrimitives.tc`** per-primitive fuzzer, which runs nightly and has already found real crashes that fifteen years of hand-written tests had not. The nightly multi-configuration build catches portability regressions that a single-configuration CI would miss. The wildcard mechanism elegantly solves the problem of platform-dependent output without requiring a separate comparison framework.

What GNU APL currently lacks — and what a future maintainer could profitably add — is:

- a fuzzer harness for the tokenizer and parser (the existing fuzzer targets primitives called with already-valid arguments, not malformed source text);
- AddressSanitizer / UBSanitizer runs in the nightly matrix;
- commit-triggered CI rather than time-triggered nightly builds;
- extending **`CONFORM_A`** conformance markers to the roughly 62 **`rval.def`** rows that do not yet have one (see "**`Running It Nightly`**" earlier in this chapter), and wiring **`INVARIANTS.apl`** into the same nightly run.

None of these are architectural changes; they are infrastructure additions that build on the existing test foundation. The foundation itself is sound.

3.13. Excursus: Interpreter Overhead, Then and Now

Compiled versus interpreted, in one sentence. A compiler reads a program once, ahead of time, and emits machine code that performs exactly the operations that program requires and no others; an interpreter reads (or re-reads) the program **while running it**, and every one of those operations is preceded by some amount of bookkeeping — decide what this token means, dispatch to the code that handles it — that a compiled program's machine code simply does not contain, because the compiler already resolved all of that ahead of time. That bookkeeping cost is "interpretation overhead." It is not the only difference between the two execution styles (a compiler can also see across statement boundaries and optimize sequences of operations together, something a naively-implemented interpreter cannot), but it is the part most people mean by the term.

Why it was a bigger problem in the 1970s. Early interpreters — timesharing BASIC, early APL implementations among them — routinely carried interpretation overhead that was large relative to the "useful" work each operation did, for a reason that has largely disappeared from modern hardware: a 1970s CPU executed one simple instruction at a time, with no instruction cache, no branch prediction, and no pipelining to hide the cost of an indirect jump. Dispatching to the right handler for a bytecode or an APL glyph — a table lookup followed by an indirect jump — was not a cheap, speculatively-prefetched branch the way it is on a modern out-of-order core; it was a comparatively expensive operation in its own right, and a scalar operation like adding two numbers did too little "real work" to amortize it. This is precisely why idiomatic APL avoided explicit scalar loops from the start: `+/A` pays dispatch overhead **once**, for the whole reduction, however large `A` is, while a hand-written scalar loop computing the same sum in an early interpreter would pay it once **per element** — the array-at-a-time programming style this book has covered since Chapter 4 was, originally, as much a performance discipline as a notational one.

What GNU APL's own numbers say today. Table 3-3 earlier in this chapter is directly relevant here, read for a different question: not which layer of the interpreter dominates, but how much of total execution is interpretation overhead in the sense just defined. `Parser.cc` plus `Prefix.cc` — tokenizing, parsing, and deciding what to execute next, the parts of the pipeline with no analogue at all in compiled code — together accounted for roughly 8 % of self-time in that profile. The rest was either genuine array/value work (moving cells, resizing ravels, looking up symbols — work a compiled implementation of the same algorithm would also have to do) or the primitives themselves. Modern hardware is one reason that number is small — the dispatch that cost real cycles on a 1970s CPU is comparatively cheap once branch prediction and instruction caches are doing their job — but it is not the only reason. `RavelType`-packed storage (§4.2.3, Chapter 4) and the per-primitive fast-path workers

Chapter 11 describes exist specifically so that a packed numeric array is processed as a tight native loop over **`int64_t`**/**`double`**, not as a sequence of individually-dispatched, individually-boxed **`Cell`** operations; that architecture is, in effect, a deliberate, modern continuation of the same discipline 1960s and 1970s APL programmers practiced by hand when they wrote **`+/A`** instead of a loop.

Could APL just be compiled, then? It has been tried — commercial and academic efforts to build APL compilers go back decades — and the gains were consistently narrower than hoped, for reasons this chapter's own numbers help explain. A compiler can only remove the overhead of deciding what to execute; it cannot make the execution of **`+/A`** itself any faster than a well-written interpreter's native **`+/`** already is, because **both** are, in the end, calling the same kind of tight native loop over array data. If interpretation overhead — the part a compiler actually eliminates — is already a modest single-digit-to-low-double-digit percentage of total runtime for idiomatic, vector-style APL code, then the very best a static compiler can do is shave that slice off, not the whole runtime; the ceiling on the win was low well before anyone tried to build the compiler. Two further obstacles made even that modest ceiling hard to reach in practice. First, APL's types and ranks are late-bound: the same variable can hold a scalar integer on one line and a nested character matrix on the next, and a compiler that cannot prove otherwise has to either fall back to runtime type dispatch anyway — reintroducing the very overhead it set out to remove — or restrict itself to a statically-typed subset of the language, which excludes most idiomatic APL. Second, **⍎** and **⌸FX** (Chapter 14) let a program construct and execute APL source text **at run time**; no ahead-of-time compiler can compile code that does not exist yet, so any APL implementation claiming full language coverage needs an interpreter on hand regardless of how much of the rest gets compiled. Where real, substantial wins over interpretation **have** materialized for array languages in more recent years, they have tended to target a different bottleneck than dispatch overhead altogether — fusing chains of array primitives so that intermediate results are never fully materialized in memory, and compiling the fused pipeline to a GPU or SIMD kernel — which is a much larger engineering undertaking than 1970s-style ahead-of-time compilation, and addresses memory traffic rather than per-token dispatch. That GNU APL remains an interpreter is, per this chapter's own measurements, not obviously leaving much of that particular kind of performance on the table.

3.14. Exercises

1. [1] Run **`make -C src test3`** and note the total number of testcase files executed and the total number of errors. Identify the line in **`src/testcases/summary.log`** that reports the final totals.

2. [2] Write a new testcase file **`src/testcases/MyRho.tc`** that tests the monadic **`⍴`** function on arrays of ranks 0 through 3 — a scalar, a vector, a matrix, and a rank-3 array. Include a **`)CLEAR`** at the top. Run the file with **`--TM 3`** to verify it passes.
3. [2] Add a testcase for **`⎕TS`** that uses the **`⍬`** wildcard to match the variable time components. Confirm that the testcase passes on two consecutive runs. Then manually replace one wildcard with a literal wrong value and confirm the test fails on the next run.
4. [2] Trace through **`DiffOut::different()`** in **`src/DiffOut.cc`** and document in a short comment what each of the eight wildcard characters matches and what it *does not* match. In particular, explain why **`²`** does not match the integer **`42`** (which it does match) differently from the way **`⍬`** matches it.
5. [3] Add a new wildcard character — say **`⁷`** — that matches zero or more APL uppercase letters (the characters in **`⎕AV`** positions 65–90). You will need to: (a) add a **`UNI_DIFF_ALPHA`** constant to **`Unicode.hh`**; (b) add the corresponding case to **`DiffOut::different()`**; (c) write a testcase that uses it. Run **`make -C src test3`** to confirm no regressions.

How to Order

If you like what you read so far then you can order the entire book. All you need for that is its title or its ISBN: **ISBN 978-3-6967-1351-5**.

A. Bookstores: Order the book at your local bookstore, in any country. This is the slow path because the book is shipped in several hops: publisher → wholesale → bookstore, which takes time.

B. Europe: You can order the book directly online from the publisher (**BoD**). Here is a direct link to the book's page in BoD's shop:

https://buchshop.bod.de/

BoD will then ship the book directly to you. **BoD** ships within Europe, but not to the US. Note that this path is not that fast either: the book is printed fresh after your order has landed at BoD (allow 2 weeks for that step).

C. USA: You can order the book online from Walmart. Here is a direct link to the book's page in Walmart's shop:

https://www.walmart.com/

Other US retailers sell the book as well. Unlike in Germany, where every seller must charge the same price for a new book, US retailers set their own prices, and these can differ considerably. It pays to compare them (e.g. by searching for the ISBN) before you order.